

C07 — Product Design: Designing for a User

1. What interviewers test

This is one of the most heavily weighted question types in product loops [Inferred](#), so the chapter is long and specific. The prompt is open-ended: "Design a product for X," "Design a feature for Y on a smartwatch," "Design something to help people do Z." There is no right answer. The interviewer is watching how you get from a vague prompt to a defensible design, and whether you can show your reasoning at each step.

Where it shows up. [Inferred](#) from the round structures candidates commonly report: a dedicated product sense or product design round at Google, Meta, Microsoft, Amazon, Apple and other companies, often 35 to 45 minutes, usually with a prompt tied to the company's own products or to a user group. On the job the same skill is what a PM does when a team has an area to work on and no solution yet: pick a user, find the problem, shape a solution, define success.

What is scored. Google publicly describes hiring on general cognitive ability, role-related knowledge, leadership and "Googleyness" [Public](#) [[V: summaries citing Google's how-we-hire page](#); [Google re:Work structured-interviewing guide](#)], and a design round shows the first two. Amazon publishes Customer Obsession as a leadership principle [Public](#) [[V: Amazon's Leadership Principles page](#), [amazon.jobs](#)]. Apple publishes platform design guidelines and a public privacy stance [Public](#) [[V: Apple Human Interface Guidelines and privacy pages](#)], and third-party guides report that its loops weigh craft and fit with the ecosystem [Inferred](#). Microsoft publishes inclusive-design principles [Public](#) [[V: Microsoft Inclusive Design toolkit manual](#)]. Scorers typically write about five things [Inferred](#): whether you structured the answer, how well you understood and chose a user, whether the problem you chose was real and important, whether your solutions were creative and feasible, and whether you reached a decision and a way to measure it.

Five observable behaviors:

1. You clarify the prompt and set a goal before you design.
2. You choose a specific user by a stated criterion.
3. You rank the user's problems and pick one with a reason.
4. You offer solutions that differ in kind, choose one, and describe the first version.
5. You define success, name the risks (including misuse and privacy), and say what would change your mind.

The failure this prevents. Six patterns end product design answers: a feature list with no user, a solution chosen before the problem, a demographic user ("busy moms"), solutions that are variations of one idea, no decision at the end, and ignoring the downside (privacy, safety, misuse, accessibility).

2. The concept from first principles

C07.1 What product sense is

Product sense is the ability to identify the right user problem, generate and choose between solutions, and define success, usually assessed with an open-ended prompt. [[product-sense]] is not taste alone, and it is not design in the sense of drawing screens. It is the chain of judgments that connects a person's situation to a thing worth building: who, what is hard for them, which of several ideas is best and why, and how you would know it worked.

It builds on the earlier chapters. The decision structure and tradeoff vocabulary come from C02 ([[tradeoff]], [[reversible-decision]]). The structuring habits come from C03 ([[mece]], the issue tree). The metrics step is C05. The user understanding is C06 ([[user-segment]], [[jobs-to-be-done]], the [[problem-statement]]). This chapter assembles them into one answer you can deliver in 45 minutes.

Why a fixed spine works. An open prompt invites wandering. A spine gives you somewhere to go next at every moment, and it makes your reasoning visible, which is what the interviewer can score. Candidates who improvise often produce good ideas that the interviewer cannot trace to a user or a problem.

What interviewers do not need. Wireframes, polished screens, or technical architecture. A rough sketch of a flow, described in words, is enough. Interviewers want product thinking, not interface design **Inferred**.

When not to use the full spine. If the prompt already fixes the user ("design an alarm clock for people who are deaf"), skip the segmentation and spend the time on the problem and the solutions. If the interviewer redirects you ("let's go straight to solutions"), follow them. If the prompt is about improving an existing product, the spine changes (C08), and for a market you do not know, it changes again (C09).

Counterexample. A candidate asked to "design an app for people who are blind to get around a city" spends twelve minutes segmenting users and listing criteria, though the prompt already named the user group. The interviewer is left with five minutes for the problem, and the solution is thin. Spend segmentation time where the prompt leaves the user open. A fixed spine is a guide to what to cover, not a quota of minutes for each step.

Hidden risks. A rigid spine can make your answer sound recited ([framework-theater]), C03). The defense is to adapt each step to the prompt, to say what you are skipping and why, and to let a good insight change the plan. A second risk is creativity theater: a surprising idea with no link to the user's problem. Interviewers reward ideas that follow from the problem.

C07.2 The answer architecture for a 45-minute round

Section 4 sets out the full spine with time budgets. In brief: clarify, set the goal, choose a user, rank the problems, generate solutions, evaluate and recommend, measure, and check the risks. Each step produces something the interviewer can write down.

C07.3 Clarify, then set the goal

Clarifying questions (three to five, about two minutes) fix the scope and the stakes. The useful kinds:

Kind	Example	Why it matters
Product and platform	"Is this a new product or a feature in an existing one? On which device?"	Constrains the solution space
Company and goal	"Why does the company want this? Growth, engagement, revenue, a mission?"	Defines what a good answer optimizes
User	"Is there a particular user in mind?"	Saves the segmentation step if one is given
Constraints	"Any limits on time, team, market or technology?"	Prevents an unbuildable answer
Success	"How will the team judge it?"	Seeds the metrics step

If the interviewer says "you decide," state your assumptions aloud: "I'll assume this is a new feature in an existing mobile app, with a goal of increasing meaningful use, and I'll flag if that changes."

Set the goal. Name what you are optimizing and for whom, in a sentence. A goal ladder (FW-C07-02) has three rungs: the company's purpose in building this, the product's goal in this area, and the user's goal. The best answers show that the user's goal and the company's goal point the same way, and say so when they do not. The goal also bounds the problem: "increase the number of people who finish a first workout" leads to different designs from "increase revenue per subscriber."

Why it works. The goal is the criterion against which you will choose a solution later. Without it, every solution is as good as another.

When not to do it at length. In a short prompt, a one-sentence goal is enough.

C07.4 Selecting a user: segmentation criteria and a defensible choice

Apply C06.2 in a minute or two: list three or four segments by need and behavior, pick one by a stated criterion, and say what you give up. Four criteria cover most prompts: size, pain intensity, reach, and fit with the company (FW-C06-02).

A short persona sentence helps the interviewer see the person: "Maya, a first-time marathon trainer, follows a 16-week plan, runs before work, and checks her wrist mid-run." Keep it to two sentences, and make every detail tie to a need or constraint. Decorative details (a hobby, a pet) add nothing.

Name your criterion out loud. "I'll focus on first-time trainers because their pain is most acute and they have the least support, and I'll leave experienced runners, who are a larger but better-served group."

When the user is given. Confirm it and move on, and say what makes this user's situation specific.

Hidden risk. Picking a segment because it is easy to imagine rather than because it matters. A good check is whether the interviewer could argue for a different segment and whether you could answer them.

C07.5 Pain points: ranking by frequency and severity

A **pain point** is a specific, observable difficulty a user meets while trying to do something, which can be ranked by how often it occurs and how much it costs them. [[pain-point]] is the unit of the problem step. Generate pain points by walking through the user's journey in stages (before, during, after, or by steps of the task) and listing what goes wrong in each (FW-C07-03).

Rank them. Use frequency (how often), severity (how costly when it happens) and the current workaround (what they do about it; a workaround is evidence the pain is real). Pick the top one or two, say why, and say what you are leaving.

Pain point	Frequency	Severity	Workaround today	Rank

Make them specific. "Running is hard" is not a pain point. "Going out too fast in the first third of a long run and fading" is. A pain point should describe something you could watch happen.

Say how you know. In an interview you cannot run research, so say whether the ranking comes from experience, from reasoning about the journey, or from what you would verify ("I'd check this with interviews, and I'd expect pacing to rank above shoe selection"). That honesty reads as judgment, not weakness.

Why it works. The problem you choose bounds the design, and the ranking shows you made a choice and can defend it.

When not to rank at length. If the prompt names the problem ("help people who forget to take medication"), spend the time on why it happens and for whom, not on a long list.

Counterexample. A candidate lists eight pain points for every user group, gives each one sentence, and ranks none. The interviewer cannot tell which problem the later solutions address. Fewer, ranked, and specific beats many and flat.

C07.6 Generating solutions that differ in kind

Interviewers want to see range and judgment. Offer three or four solutions that differ **in kind, not in degree**: not "a small widget, a bigger widget, a full app," but different ways of solving the problem. FW-C07-04 lists a spectrum:

Kind	Idea	Example for "runners go out too fast"
Eliminate	Remove the cause of the pain	A plan that sets pace automatically each mile
Automate	Do the work for the user	The device adjusts targets from heart rate in real time
Assist	Give information or a nudge at the right moment	A tap on the wrist when pace exceeds the target
Reframe	Change what the user is trying to do	Train by effort level rather than pace
Connect	Use other people or systems	A friend paces with you remotely

For each, say in one or two sentences how it works for the user, why it addresses the pain, and what it costs. Do not describe screens. Describe moments: "at mile three, her wrist taps once if she is 10% faster than her plan."

Use constraints as a source of ideas. The platform (a watch has seconds of attention), the context (hands busy), and the user's resources (no time to learn) all narrow and sharpen solutions.

Technology is a tool, not a solution. Mention machine learning or a new sensor only where it solves a named problem, and say what happens when it fails (C24 and C25 for AI features).

Why it works. Range shows you are not attached to the first idea, and the contrast makes your choice a judgment.

When not to use many options. If time is short, two well-contrasted options are better than four thin ones.

C07.7 Evaluating, recommending and phasing

Compare options against criteria drawn from your goal, using a short tradeoff table (FW-C02-03, FW-C07-05). Typical criteria: impact on the goal, effort, risk, reach, and fit with the company's strengths. Do not hide behind scores; say the sacrifice: "I choose the haptic pacing, which gives up adaptive plans for the first version."

Recommend one. Describe the first version in four to six steps as a flow of moments, not as screens: what the user does, what the product does, and what the user feels. Then say what comes next: a second version and a third, in order, with the reason for the order.

Say what would change your mind. Name a result that would make you pick a different option (C02's closing habit).

Why it works. A recommendation with a stated cost shows decision-making, which is the signal that separates strong from baseline answers.

When not to phase. If the interviewer asks only for the idea, a sentence on what comes later is enough.

C07.8 Success metrics and guardrails

Use the template from C05 (FW-C05-06): the goal, the user behavior, one primary metric defined precisely, two or three inputs, guardrails with thresholds, a counter metric, and how you would measure. In a design round you have about five minutes, so be compact: one primary metric with a definition, one input, one guardrail, one counter metric, and a sentence on how you would learn whether the design causes the effect.

Match the metric to the problem you chose, not to a generic metric. A pacing feature is measured by completed runs at planned pace, not by app opens.

C07.9 The responsible-design check: trust, safety, privacy, misuse

Every product can hurt someone. End your answer by saying how this one could, and what you would do about it. FW-C07-06 gives a checklist. The questions that matter most:

- **Misuse:** how could a bad actor use this against another person? (Location, messaging and sharing features need this question first.)
- **Vulnerable users:** children, older adults, people in crisis or in unsafe relationships.
- **Privacy and consent:** what data does this use, who sees it, can the person say no, and is the default the protective one?
- **Safety:** could the design lead to physical harm (a distracted walker, a runner pushed beyond a safe effort)?
- **Wellbeing:** could it encourage compulsive use or anxiety?
- **Accessibility and inclusion:** who cannot use it as designed?
- **Failure:** what happens when it is wrong (a wrong alert, a missed signal)?

State two or three that matter for this product, with a concrete design response ("sharing is off until each person opts in, with a visible indicator and a one-tap pause"). Naming the risk without a response is half an answer, and a risk list with no link to the design is filler.

Why it works. It shows maturity, and at several companies it is a stated value (privacy at Apple, inclusive design at Microsoft, trust and integrity questions at social companies [Inferred](#)).

When not to dwell. Do not let it consume the answer; two minutes at the end is right.

C07.10 Designing across surfaces and form factors, including hardware-software integration

The same problem leads to different designs on different surfaces. The surface decides how much attention you have, how the user inputs, and what the product can sense. FW-C07-07 gives the lens.

Surface	Attention	Input	Typical strengths	Typical limits
Phone	Focused, short sessions	Touch, voice	Rich interaction; always with the user	Hands and eyes busy in motion
Watch	Glance of a few seconds	Taps, crown or buttons, voice	On the body; sensors; haptics	Tiny screen; battery; little text
Voice speaker or assistant	Ambient, eyes free	Speech	Hands-free	No screen; errors in noise; privacy at home
TV	Shared, leaned back	Remote	Group viewing; big screen	Slow input; poor for reading
Car	Safety-critical	Voice, controls	Hands-free context	Distraction risk; must be glanceable
Physical device	Whatever the form allows	Buttons, sensors	Tangible; no app needed	Hard to change after shipping; cost of hardware

Hardware-software integration. Where a company makes the device and the software, the design can use sensors, battery and processing together: a watch that senses heart rate and also tells the user what to do about it. Questions to ask: what does the hardware sense, what can be computed on the device (which helps privacy and speed), what the battery allows, how it works with the user's other devices, and what cannot be changed after the product ships. A mistake in software can be patched; a mistake in hardware often cannot (C17.3).

Continuity. Many problems span devices: start on a phone, continue on a watch or speaker. Mention the handoff and what stays consistent.

When not to use the lens. If the prompt does not name a device, choose one with a reason, and spend one sentence on it.

C07.11 Worked designs across product types

Section 5 gives four complete worked designs: a consumer commuting product, a wearable feature, a navigation problem in a public space, and a voice-only experience. Section 11 adds seven interview questions with full model answers, including two set on devices made by Apple and one on a maps product.

3. Frameworks

Seven frameworks, set out below. The segment scorecard (FW-C06-02), the tradeoff table (FW-C02-03), the success-metric template (FW-C05-06) and the pacing skeleton (FW-C03-07) are reused, not repeated.

FW-C07-01 · The design answer spine · Original

- **Inputs:** a design prompt and 35 to 45 minutes.
- **Outputs:** an answer with eight visible steps: clarify, goal, user, problems, solutions, recommendation, metrics, risks.
- **Assumptions:** the interviewer will answer clarifying questions and may redirect.
- **Tradeoffs:** structure makes the reasoning scorable and can make the answer sound mechanical.
- **Failure modes:** equal time on every step; skipping the decision; reciting the steps without adapting them to the prompt.
- **Worked example:** the commuting-product design in Worked example 1, and the full model answers in section 11.

Steps: **Clarify** → **Goal** → **User** → **Problems** → **Solutions** → **Recommend** → **Measure** → **Risks**. The time budget is in section 4.

FW-C07-02 · The goal ladder · Original

- **Inputs:** a prompt and what you know of the company.
- **Outputs:** a three-rung statement: the company's purpose, the product goal in this area, and the user's goal, with a note on where they conflict.
- **Assumptions:** the company's purpose can be reasoned about from public information.
- **Tradeoffs:** a clear goal sharpens choices and can exclude interesting ideas.
- **Failure modes:** a goal that is only a business metric; a goal the user would not recognize; pretending that goals never conflict.
- **Worked example:** for a smartwatch feature, the company wants the device to be part of daily life, the product goal is regular healthy use, and the user's goal is to finish a first marathon without injury (IQ-C07-01).

Rung	Question
Company	Why would the company build this?
Product	What outcome in this area should improve?
User	What is the person trying to achieve?
Conflict	Where might these pull apart?

FW-C07-03 · The pain-point matrix · Original

- **Inputs:** a user and the stages of their task.
- **Outputs:** five to seven pain points, each rated for frequency, severity and workaround, with the top one or two chosen and a stated reason.
- **Assumptions:** you can reason about the user's journey or have evidence from research.
- **Tradeoffs:** ranking forces a choice and rests on judgment when evidence is thin.
- **Failure modes:** vague pain points; no ranking; picking the one that suits a pet solution.

- **Worked example:** the commuting pain points in Worked example 1, ranked, with schedule unreliability chosen.

Steps: (1) list the stages of the task; (2) list what goes wrong at each; (3) rate frequency, severity and workaround; (4) pick the top one or two and say why; (5) say what evidence would change the ranking. The table is in C07.5.

FW-C07-04 · The solution spectrum · Original

- **Inputs:** the chosen pain point and the constraints.
- **Outputs:** three or four solutions of different kinds (eliminate, automate, assist, reframe, connect), each with how it works, why it helps and what it costs.
- **Assumptions:** the solution space is wider than the first idea.
- **Tradeoffs:** range versus depth in limited time.
- **Failure modes:** variants of one idea; solutions with no link to the pain point; technology-first ideas.
- **Worked example:** pacing solutions for first-time marathon trainers (IQ-C07-01), and the movement nudges in Worked example 2.

The kinds are listed in C07.6.

FW-C07-05 · The option evaluation grid · Original

- **Inputs:** the solution options and the goal.
- **Outputs:** a comparison on impact on the goal, effort, risk, reach and fit, a chosen option, a phased plan, and the sacrifice stated.
- **Assumptions:** criteria can be drawn from the goal; rough ratings are enough.
- **Tradeoffs:** a grid is quick and can imply false precision (see FW-C02-03).
- **Failure modes:** scoring that justifies the favorite; no stated sacrifice; no phasing.
- **Worked example:** the station-navigation grid in Worked example 3.

Option	Impact on goal	Effort	Risk	Reach	Fit
A					

Recommendation line: "I recommend B for the first version, which gives us ___ by giving up ___; next I would add ___; I would switch to A if ___."

FW-C07-06 · The responsible-design check · Original

- **Inputs:** the chosen design.
- **Outputs:** the two or three most relevant risks from a standard list, each with a concrete design response, and a launch condition or kill criterion where needed.
- **Assumptions:** you can anticipate misuse and vulnerable users by reasoning about the product's data and reach.
- **Tradeoffs:** safeguards cost friction and adoption; no safeguards cost trust and safety.

- **Failure modes:** a list of risks with no design response; defaults that favor growth over protection; ignoring bad actors.
- **Worked example:** the family location-sharing answer in IQ-C07-05, and the pre-mortem in C02 (Worked example 4).

Check	Question	Typical response
Misuse	How could someone use this against another person?	Consent from each person, visible indicators, easy exit
Vulnerable users	Children, older adults, people in unsafe situations?	Protective defaults, extra controls
Privacy and consent	What data, who sees it, can they decline?	Minimize data, on-device processing, opt-in
Safety	Could it lead to physical harm?	Limit use while moving, cap intensity
Wellbeing	Could it encourage compulsion or anxiety?	No streak shaming, quiet defaults
Accessibility	Who cannot use it?	Multiple input and output modes
Failure	What happens when it is wrong?	Graceful fallback, clear uncertainty

FW-C07-07 · The surface and form-factor lens · Original

- **Inputs:** a problem and a device or context.
- **Outputs:** the constraints that the surface imposes (attention, input, sensing, battery, connectivity, privacy in public), and a design that fits them.
- **Assumptions:** the surface is known or can be chosen with a reason.
- **Tradeoffs:** designing for one surface can fit poorly on another.
- **Failure modes:** porting a phone design to a watch; ignoring battery; forgetting what the device can sense.
- **Worked example:** the wearable movement nudges in Worked example 2, and the voice cooking assistant in Worked example 4.

The table is in C07.10. Steps: (1) state the surface and why; (2) list its constraints; (3) say what it makes easy; (4) design the moments around those; (5) say how it connects to the user's other devices.

4. Answer architecture

The design answer, for a 45-minute round

The structure (FW-C07-01): Clarify, Goal, User, Problems, Solutions, Recommend, Measure, Risks.

Time budget

Step	Minutes	What to produce
Clarify and set the goal	4	Three to five questions; a stated goal and assumptions
Choose the user	5	Three or four segments; one chosen with a criterion; a two-sentence persona
Rank the problems	6	Five to seven pain points; one or two chosen with reasons
Generate solutions	8	Three or four solutions of different kinds
Evaluate and recommend	7	A tradeoff grid; one recommendation; the first version as a flow; what comes next
Metrics	5	Primary metric with definition, an input, guardrail, counter metric
Risks and wrap-up	5	Two or three risks with design responses; what would change your mind
Slack for interviewer questions	5	

Pacing rules. Treat the interviewer's questions as steering; follow them and rejoin the spine. If you run behind, compress the solutions to two and the metrics to a sentence, and do not skip the recommendation. If the prompt gives a user, move four minutes from the user step to the problems and solutions.

What to clarify first. The product and platform, the goal, whether a user is given, and any constraint. Ask two or three, and state assumptions for the rest.

Where to signpost. After clarifying: "I'll cover the user, their problems, solutions, a recommendation, and success measures." Before choosing a user: "I'll pick a segment by pain and reach." Before recommending: "My deciding criterion is X."

Where to state tradeoffs. In the user choice (what you give up), in the problem choice (what you leave), in the recommendation (the sacrifice), and in the risks (what safeguards cost).

How to close. Two sentences: "I recommend X for [user], to solve [problem], and I'd measure success by Y. The main risk is Z, which I'd address with W; I'd change my mind if V."

Type: the short version (20 to 25 minutes)

Keep every step and halve the time: one user, one problem, two solutions, a quick recommendation, one metric, one risk. The recommendation and the risk are the steps to protect.

Type: the user is given

"Design X for [user group]." Use the saved minutes for deeper problems (two levels of "why") and for solutions that fit the specific situation. Say what makes this user's situation different from the average user's.

Type: the surface is given

"Design Y on a smartwatch / for voice." Spend the first problem minutes on the surface's constraints (FW-C07-07). Describe moments, not screens. Say what the device senses and what the battery and attention allow.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"Who is the user?"	You skipped or blurred the user	Name a specific segment and your criterion
"Why that problem?"	Your ranking is unclear	State frequency, severity and workaround
"What else could you do?"	They want range	Offer a solution of a different kind
"How would you know it works?"	They want a metric	Use the C05 template
"What could go wrong?"	They want the downside	Misuse, privacy and failure, with a response
"Let's go deeper on the first version"	They like it	Describe the flow, edge cases and what you would cut

5. Worked examples

All four examples are *Illustrative*: the products are fictional and every figure is an invented assumption. Each follows the spine in FW-C07-01 and is written as notes you could say aloud. A spoken answer is shorter than these notes; the point here is to show every step and the reasoning behind each choice.

Worked example 1 — Consumer: a feature for people who commute by public transport (Illustrative)

Setting: a transit-planning feature of the kind offered in maps products such as Google's. No fact about any real product is claimed.

The prompt. "Design a product for people who commute by public transport."

Clarify and goal. I'd ask three things: is this a new product or a feature in an existing app, on which device, and what does the company want? Suppose the answer is "a feature in an existing transit-planning app on phones, in a mid-sized city, and the company wants the app to be part of the daily commute." The goal ladder (FW-C07-02): the company wants daily use; the product goal is that a commute goes the way the traveler planned; the user's goal is to arrive on time without wasting time. Conflict to watch: more notifications raise usage and lower trust.

User. Three segments by need and behavior: daily commuters with a fixed arrival time and at least one transfer; occasional riders such as visitors and event-goers; commuters with flexible hours. I choose the first. Pain intensity is high (a late arrival has a cost), the trips repeat about ten times a week, the app already reaches them, and the company's real-time

data fits their problem. I give up occasional riders, who are numerous but whose trips carry little at stake, and flexible commuters, who feel the pain less. Persona: Dana travels forty minutes each way with one transfer and has a 9:00 stand-up (invented).

Pain points (FW-C07-03). Walking the journey in stages:

Pain point	Frequency	Severity	Workaround today	Rank
Not knowing until she reaches the platform whether her usual train is late	About 3 trips in 10 are delayed by five minutes or more (invented)	High when a transfer is at risk	Checks the app several times; leaves 15 minutes early every day	1
Missing a tight transfer after a delay	About 1 trip in 10 (invented)	High: a 15-minute extra wait	Takes an earlier train than needed	2
Crowded platform at peak	Daily	Medium	Stands at a different platform end	3
Getting from the station to the door	Daily	Low to medium	Walks; sometimes late	4
Finding a seat	Daily	Low to medium	Stands at the usual door	5
Fare and ticket friction	Weekly	Low	Monthly pass	6
Boredom	Daily	Low	Podcasts	7

I choose the first two, which share a cause: the traveler cannot tell in time whether to leave earlier. The workaround is the evidence. Leaving 15 minutes early on about 21 working days is roughly five hours a month (invented), paid to guard against a delay that happens on three trips in ten. I'd verify the frequencies in interviews and the app's data, and I'd expect them to rank above crowding and seats.

Solutions that differ in kind (FW-C07-04).

Kind	Solution	How it works
Eliminate	Hold the connection	The operator holds the connecting train when a feeder train is late. Outside the app's control
Automate	Reroute and rebook	When a delay is detected, the app picks another route and rebooks the ticket
Assist	Latest-safe-departure alert	"Leave by 8:12 to make your 9:00." The time updates as delays develop
Reframe	Plan by arrival window	The traveler sets "be there by 9:00" and the app chooses the train, instead of the traveler choosing a train
Connect	Rider delay reports	Riders on the same train report conditions

Evaluation (FW-C07-05).

Option	Impact on goal	Effort	Risk	Reach	Fit
A. Hold the connection	High	High (needs the operator)	High (not ours to control)	High	Low
B. Reroute and rebook	High	High	Medium to high (wrong reroute, payment)	Medium	Medium
C. Latest-safe-departure alert	Medium to high	Low to medium (uses the existing real-time feed)	Low to medium (false alarms)	High	High
D. Arrival-window planner	Medium	Medium	Medium (changes a habit)	Medium	Medium to high
E. Rider delay reports	Low to medium	Medium	Medium (spam; thin data early)	Low until density builds	Medium

Recommendation. "I recommend C for the first version, which gives commuters their buffer time back by giving up automatic rerouting; next I would add rerouting for the cases where the alert is followed and the transfer is still missed; I would switch to B sooner if interviews show that people want the app to decide for them." The deadline the user sets (from D) is the one input C needs, so C borrows the useful part of D.

The first version as moments: (1) One-time setup: the user enters the usual route and the arrival deadline. (2) On a weekday at 7:52 a quiet notification says "Leave by 8:12; your train is running four minutes late and your transfer still looks safe." (3) If the transfer is at risk, the message changes to "Leave by 8:05, or take the 8:20 via Central." (4) One tap starts the trip. (5) On arrival, a single optional tap: "Did this help?"

Metrics (FW-C05-06, compact). Primary: the share of commute days on which the user's trip ends by the deadline they set, for alert users against a holdout group, so the metric compares like with like (the trip-end signal comes from the app's own trip tracking, which avoids needing continuous location). Input: the share of alerts followed by a departure within five minutes of the suggested time. [[guardrail-metric]]: false-alarm rate (alerts that told the user to leave earlier when the train ran on time), below 10% (invented threshold), and a monthly notification-mute rate below 5% (invented). [[counter-metric]]: the average buffer users add, because if it grows, the alert may be teaching people to over-leave.

Risks. Location history can reveal home and work, so commute patterns stay on the device. Alert fatigue destroys trust fast, so alerts fire only when the recommended departure time changes by more than a few minutes. When the real-time feed is wrong the alert is wrong, so the message states the confidence ("looks safe," "tight") rather than a false certainty. I would change my mind if the holdout comparison showed no difference in on-time arrivals, since then the alert would be informing people without helping them.

Worked example 2 — Wearable: a smartwatch feature to help desk workers move more (Illustrative)

Setting: a smartwatch feature of the kind Apple and several other companies make. No fact about any real product is claimed.

The prompt. "Design a smartwatch feature that helps desk workers move more during the day."

Clarify and goal. The feature is in an existing watch and phone ecosystem. The watch senses motion and heart rate, and can read the calendar with the wearer's permission. Goal ladder: the company wants the watch to be worn and used daily; the product goal is more regular movement breaks; the user's goal is to feel better by the end of the workday without losing focus. Conflict: every extra buzz that raises engagement costs trust, and the user also wants fewer interruptions in meetings.

User. By behavior: workers in back-to-back meetings, who sit for two to three hours at a stretch without noticing; deep-focus workers, who sit long by choice and resent interruptions; and hybrid workers with flexible days. I choose the first. Their sitting is structural (the calendar causes it), predictable (the product can see the blocks), and they cannot easily interrupt themselves in a meeting. I give up deep-focus workers, for whom an interruption is costly and the benefit is smaller. Persona: Sam has meetings from 9:00 to 12:30 most days, with no gap (invented).

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
Sitting through a long meeting block without noticing	Daily	Medium (stiffness, fatigue)	Takes calls standing when possible	1
A generic hourly "stand up" prompt arrives mid-presentation	Daily	Medium (embarrassing, then ignored)	Disables the reminder	2
No gap in the calendar to move	Several days a week	Medium	Declines meetings	3
The prompt says "stand" and the wearer cannot	Rare overall; for those affected, every time	High	Turns the feature off	4
Feeling judged by streaks and rings	Weekly	Medium (guilt)	Ignores the watch	5
Forgetting after returning to the desk	Daily	Low	None	6

The first two are one problem seen from two sides: reminders do not know what the person is doing, so they arrive when the person cannot act on them. The fourth is rare in the population and severe for the people it affects, so it becomes a design requirement rather than a ranked problem: the wording and the options must work for everyone (the accessibility row in FW-C07-06).

Surface lens (FW-C07-07).

Constraint	What it means here
Attention	Three seconds. One tap and one line of text
Input	A tap or a dismiss; no typing
Sensing	Wrist motion shows long stillness; the calendar shows meeting blocks
Battery	Motion sensing is cheap; continuous heart-rate and location sampling are expensive and not needed
Privacy	Calendar content stays on the device; only free-or-busy is used
Continuity	The phone holds settings and history; the watch acts in the moment

Solutions that differ in kind.

Kind	Solution
Eliminate	Suggest that meetings end five minutes early, as a prompt to the person who organizes them (a calendar change, not a watch feature)
Automate	Detect and credit movement automatically, with no prompt at all
Assist	A tap at the end of a long meeting block with "Two minutes to stretch or walk?"
Reframe	Count any movement, such as a walk to the printer, rather than only standing
Connect	Invite a colleague to a short walk

Recommendation. "I recommend the end-of-block tap, with the reframe that any movement counts. It gives the wearer a prompt at a moment they can act on, by giving up the simplicity of a fixed hourly schedule. Next I'd add a weekly view of which time slots they accept, and later the colleague invitation. I'd pick automatic crediting alone if people turn out to want no prompts."

The first version as moments: (1) A meeting block ends and the watch has sensed about 90 minutes of stillness (invented). (2) One tap on the wrist and the line "Two minutes to stretch or walk?" with "Later" and "Not today." (3) If the watch records two minutes of movement in the next ten minutes, a quiet confirmation tap follows. (4) After three dismissals for the same time slot, the nudge stops for that slot and asks once whether to change it.

Metrics. Primary: the share of nudges followed within ten minutes by at least two minutes of recorded movement. Input: the share of nudges delivered while the wearer is not in a meeting, which should be above 95% (invented). Guardrails: the monthly rate at which

wearers turn the feature off, below 3% (invented); the battery impact, below 1% of a day's charge (invented). Counter metric: the share of nudges dismissed within a second, which signals annoyance even when the feature stays on.

Risks. Wellbeing: no streak shaming, no red "failed" states, and quiet defaults. Accessibility: the wording says "move" or "stretch" rather than "stand," and the wearer can set a seated mode. Privacy: calendar content never leaves the device. I'd change my mind if the movement-following rate were high but the disable rate also rose, since that would mean the feature works for some people and costs others.

Worked example 3 — Public space: helping travelers change trains in an unfamiliar station (Illustrative)

The prompt. "Design a product that helps people find their way around a large train station."

Clarify and goal. I'd ask whether the product is an app on a phone and whether the station operator is a partner. Assume a phone app from a ticketing company, no control over the station's physical layout, and a goal of making the ticket app useful on the day of travel. Goal ladder: the company wants the app to matter at the moment of travel; the product goal is that travelers reach their platform in time; the user's goal is to make the connection without stress. Conflict: the app cannot change the station, only what the traveler knows.

User. By need: first-time transfer travelers with a 10 to 15 minute connection and luggage; regular commuters who know the station; travelers who need a step-free route (wheelchair, stroller, heavy bags); visitors who are leaving the station for the city. I choose the first. The stakes are a missed connection, the situation is unfamiliar and time-limited, and a ticketing app reaches them before they arrive. The step-free traveler is an extreme user (see the mental models below): their needs are sharper versions of everyone's, so I'd design the route so that a step-free option exists from the start, and treat their segment as the next one, not as an afterthought. I give up regular commuters, who need nothing, and visitors, who have time.

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
The way from the arrival platform to the departure platform is unclear when the traveler gets off	Every transfer	High (minutes matter)	Follows the crowd; asks staff	1
The train stops at the far end from the stairs to the next platform	Often (invented)	High	Runs	2
Positioning fails indoors, so a map dot jumps or freezes	Every indoor use	Medium	Gives up on the map	3
Signs assume the traveler already knows the platform number	Often	Medium	Waits for the board	4
Step-free routes are hard to find	For those who need them, every time	High	Asks staff; takes the long way	5

Pain point	Frequency	Severity	Workaround today	Rank
Platform changes are announced late	Occasional	High when it happens	Listens to announcements	6

I choose the first two, which share a cause: the traveler has to solve a navigation problem at the moment of arrival, with no knowledge of where the exit is relative to where the train stopped.

Solutions that differ in kind.

Kind	Solution
Eliminate	Better signs and earlier platform announcements (the operator's action; I'd recommend it, not build it)
Automate	Indoor turn-by-turn with beacons, rerouting live as the traveler moves
Assist	An offline route in landmarks and photos ("up the stairs, turn left past the bakery"), loaded when the ticket is bought
Reframe	Tell the traveler, at the first station, which part of the train to board so they get off nearest the stairs to the next platform
Connect	Booked or on-demand help from station staff, mainly for step-free travelers

Evaluation.

Option	Impact on goal	Effort	Risk	Reach	Fit
A. Signs and announcements	High	Not ours	n/a	High	Low
B. Indoor turn-by-turn	High	High (beacons per station)	Medium to high (outages, wrong dot)	Low at first (few stations)	Medium
C. Offline landmark route	Medium to high	Low to medium (one route per station pair)	Low (static, works with no signal)	Medium to high	High
D. Where to board	High	Medium (needs train formation data)	Medium (the data can change)	Medium	High
E. Staff help	Medium	Medium	Low	Low to medium	Medium

Recommendation. "I recommend D together with C for the first version, for the busiest stations, which gives travelers a short, reliable route and a shorter walk by giving up live positioning. Next I'd add step-free versions of each route, and then live indoor guidance in stations that install beacons. I'd bring B forward if user tests showed that people do not trust a static route." The two pieces reinforce each other: boarding at the right place shortens the route, and the route tells them what to do when they step off.

The first version as moments: (1) At purchase, the app downloads the transfer guide for this journey, since stations often have poor signal. (2) At the origin, a message says "Board the middle carriages; they stop beside the stairs for platform 9." (3) On arrival, the screen shows three photos and one line each. (4) A "step-free route" switch shows the alternative with its extra time. (5) If the platform changes, a notification replaces the route and says so.

Metrics. Primary: the share of transfers completed with at least three minutes to spare among travelers who opened the guide, compared with a staggered rollout across stations rather than comparing users who chose to open it with those who did not (the first group would differ in motivation). Input: the share of guide users who open the boarding advice before the first leg. Guardrails: the rate at which the boarding advice is wrong, because train formations change, below 5% (invented); and offline load success, above 99% (invented). Counter metric: phone use while walking at the station, because a guide that makes people stare at a screen creates a safety problem; the design answers it with large type and short audio cues.

Risks. Failure: if formation data is uncertain, the app shows the general route and says it is unsure, and does not give a confident wrong carriage. Safety: glance-only design and no demand to hold the phone up. Accessibility: the step-free route is built in, not added later. I'd change my mind if tests showed that the traveler's problem was mainly announcements arriving late, which would be an operator conversation and not a product.

Worked example 4 — Voice: a cooking assistant with no screen (Illustrative)

The prompt. "Design a voice-only assistant that helps people cook from a recipe."

Clarify and goal. The surface is given: a smart speaker in the kitchen. Goal ladder: the company wants the speaker to be useful in a daily routine; the product goal is that cooks finish recipes by voice; the user's goal is to make the meal without wrecking it or their phone. Conflict: the speaker is always listening in the home, so trust is part of the product.

User. By situation: cooks following a recipe for the first time (hands busy, high attention); experienced cooks who use a recipe as a rough guide; people cooking for a group on a deadline. I choose the first. They need the most help, they follow steps in order, and their hands are messy, which is exactly when a screen fails. I give up experienced cooks, who want less guidance, and group cooks, whose problems include coordinating dishes. Persona: Lee is making lentil soup for the first time with wet hands, with the extractor fan on (invented).

Pain points (reasoned from the situation; I'd confirm by watching someone cook).

Pain point	Frequency	Severity	Workaround today	Rank
Losing the place in the recipe after an interruption	Most cooking sessions	High (a missed step spoils the dish)	Re-reads the whole page; puts a finger on the step	1
Touching a phone screen with messy hands	Every session	Medium	Wipes hands; the screen times out	2
Running two timers and forgetting which is which	Several recipes a week	High	A kitchen timer plus memory	3

Pain point	Frequency	Severity	Workaround today	Rank
Not knowing the ingredients are all there before starting	Often	Medium	Reads the list at the start and again	4
Unclear steps ("cook until done")	Often	Medium	Searches for a video	5

I choose the first three, which are all problems of memory and hands.

What voice changes (FW-C07-07). No screen means every output must be said, so it must be short and in a sensible order. Speech recognition fails in noise, so important actions need a confirmation. The speaker cannot see the cook, so it cannot tell whether a step is done. Users cannot scan a spoken list, so the assistant reads in small chunks. The cook cannot say "tap," but can say "next," "repeat," "go back" and "how long?"

Solutions that differ in kind.

Kind	Solution
Eliminate	The assistant holds the place: it knows which step the cook is on, so interruptions cannot lose it
Automate	Named timers started automatically from the recipe ("pasta, nine minutes")
Assist	Read the ingredients three at a time and ask for a "ready" before the first step
Reframe	Describe the cue instead of the time ("until the onions are soft and clear")
Connect	Hand off to a video call with a relative who knows the recipe

Recommendation. "I recommend the place-holding assistant with named timers and cue-based steps for the first version, which solves the memory problems by giving up recipe discovery and any visual content. Next I'd add the ingredient check and pantry awareness. I'd test a screen companion if cooks keep asking to see the steps." The central design decision is that the assistant holds state, so the cook never has to.

A moment of the first version (invented dialogue):

Lee: "Start the lentil soup." **Assistant:** "Lentil soup: six steps, about forty minutes. Eight ingredients; I'll read three at a time. Say 'ready' when you have them." ... **Lee:** "How long is left on the onions?" **Assistant:** "Onion timer: four minutes. The soup timer starts at step four."

When a timer is set, the assistant confirms in plain words ("Pasta timer, nine minutes, started") so that a mishearing is caught immediately. "Next," "repeat" and "go back" always work, and after a long silence the assistant says nothing, because it cannot tell silence from a busy cook.

Metrics. Primary: the share of started recipes completed by voice alone, without the user switching to a phone or a screen mid-recipe. Input: the share of sessions in which the cook uses "repeat" or "where was I," which shows that the state-holding is being used. Guardrails:

recognition-error rate per session ("Sorry, I didn't get that"), and timer accuracy. Counter metric: abandonment mid-recipe, which could rise if the assistant is more annoying than a printed page.

Risks. Privacy: the speaker listens in the home, so the wake word is the only trigger, audio stays on the device until it is spoken, and there is a visible indicator and a mute control. Safety: the assistant gives times and cues, never claims food is safe to eat ("chicken is cooked") from time alone, and says to check with a thermometer. Accessibility: speech-impaired or accented speakers may be misrecognized, so a simple fallback (a button on the speaker) matters. I'd change my mind if the testing showed that cooks wanted to see the steps, which would move this to a screen-and-voice product.

6. Baseline, Strong, Standout

Prompt: "Design a product that helps older adults stay in touch with their families."

Baseline answer

"I'd build an app for seniors with a simple interface. It would have video calls, photo sharing and big buttons. I'd make the font large and add reminders to call family."

What is wrong: "older adults" is a demographic, not a user. There is no problem, no ranking and no alternative to the first idea. The design assumes the person has a phone and wants an app. There is no decision, no metric, and nothing on risk.

Strong answer

"I'd first ask whether this is a new product and who the company is, and assume a new product from a company with consumer hardware and software experience, and a goal of strengthening family contact. 'Older adults' includes people who are active with smartphones and people who have never used one. I'd segment by situation: those who live alone and are comfortable with a phone, those living alone who are not, and those in care homes. I'd choose people who live alone and are not comfortable with phones, because they are most at risk of isolation and are least served by current products.

"Their problems, ranked by how often and how costly: starting a call is hard (several steps and small buttons), they miss calls because the phone is in another room, family members cannot tell when it is a good time to call, and settings go wrong and nobody can fix them remotely. I'd pick starting and receiving a call, because it happens every day and it is the root of contact.

"Solutions that differ in kind: a dedicated device with one button per family member; an app on the person's existing phone with a simplified home screen; a family-managed service that sets everything up from the family's phone; and a scheduled weekly call with reminders. I'd recommend the dedicated device for the first version, since it removes the learning, and I'd measure success by the number of two-way calls per week between each pair. I'd flag risks: cost and setup, and scams."

Standout answer

Everything in the Strong answer, plus:

"The buyer and the user are different people. The adult child buys and sets it up, and the older parent uses it. So there are two users with different goals and I'd design for both: the parent needs calls to be effortless and respectful, and the adult child needs to know it works without checking on the parent. That difference decides the setup design: the child configures it remotely, and the parent never sees a settings screen.

"I'd watch for the dignity problem. A product that reports on the parent's activity (when they woke, whether they moved) can feel like surveillance. So any status sharing is the parent's choice, shown to them, and off by default, and I'd say that out loud because it is the line between a connection product and a monitoring product.

"The most serious risk is that older adults are frequent targets of scam calls. A device that makes calling easy also makes receiving calls easy, so by default only approved contacts ring through, and unknown callers go to a quiet inbox that the family can review with the parent's permission.

"I'd choose the dedicated device partly because of the surface. A person with no smartphone habit has nothing to learn with a button, but a hardware device is hard to change after shipping, so I'd run the first version as a prototype with a few families before committing to manufacturing.

"Metric: weekly two-way contact per pair, not minutes of use, since a longer call that the parent finds tiring is not a win. A counter metric is the parent's reported sense of obligation, which I'd check in short interviews. I'd change my mind if parents turned out to want the call on their own phone, in which case the product is a simplified layer and not a device."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Replaces a demographic with a situation and a criterion	Separates the buyer from the user and designs for both
Ranks problems and picks one	Names the dignity and consent line and the main misuse risk (scams), with a default that answers it
Offers solutions of different kinds and recommends one	Links the surface choice to the problem and flags what cannot be changed after shipping
Gives a metric	Chooses a metric that cannot be gamed by longer or more frequent use, and says what would change the recommendation

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Google [as of 2026-10]. Google has long published a stated philosophy that starts with focusing on the user Public [V: Google, Ten things we know to be true], and it publishes design guidance for its products Public [V: Material Design and Google's published design principles (listing on principles.design)]. Candidates commonly report product-design prompts that use Google's own products, often at consumer scale and across many countries Inferred. Practical consequence: choose a specific user, and say how the design would work for people with older phones, poor connections or different languages. Appendix A has briefs on eight Google products to practice against.

Company lens: Amazon [as of 2026-10]. Amazon publishes Customer Obsession as a leadership principle Public, and former leaders describe writing a press release and a list of customer questions before building, known as working backwards Public [V: Amazon working-backwards material (AWS re:Invent 2022 deck; Fast Company)]. Candidates commonly report that design answers are expected to start from the customer and a concrete problem Inferred. Practical consequence: say the customer's problem in the customer's words, and be ready to write the one-paragraph announcement that would make that customer care.

Company lens: Meta [as of 2026-10]. Meta publishes a mission centered on connecting people Public [V: Meta careers page: build the future of how people connect]. Candidates commonly report that product-sense rounds expect a specific user, a quantified problem, and follow-up questions on how you would know the design works and on risks such as safety and misuse Inferred. Practical consequence: pair each user claim with a way to size it, and prepare a real answer for misuse of any feature that connects people.

Company lens: Microsoft [as of 2026-10]. Microsoft publishes inclusive-design principles and a toolkit Public [V: Microsoft Inclusive Design toolkit manual (Microsoft Design)], and its products often serve a buyer, an administrator and a daily user who are different people Inferred. Practical consequence: say whose problem you are solving, and treat people with permanent, temporary and situational limits as a source of design ideas.

Company lens: Apple [as of 2026-10]. Apple publishes human-interface guidelines for its platforms and a public position on privacy Public [V: Apple Human Interface Guidelines (developer.apple.com) and privacy pages], and it designs hardware and software together Public. Third-party guides report that design prompts at Apple often involve its own devices and weigh craft and fit with the user's other devices Inferred. Practical consequence: name what the hardware senses and what stays on the device, describe moments rather than screens, and say how the design works with the user's other devices.

Company lens: Netflix [as of 2026-10]. Netflix's published culture material emphasizes judgment and context Public [V: Netflix culture memo, June 2024]. Candidates commonly report design prompts about personalization and viewing habits that expect a behavioral measure of success Inferred. Practical consequence: tie each design to a behavior you can observe, and say what you would test before building.

Company lens: Stripe [as of 2026-10]. Stripe publishes a mission about growing online commerce and is widely described as emphasizing documentation and developer experience Public [V: Stripe newsroom (mission to grow the GDP of the internet); the documentation reputation is widely described and not sourced]. Prompts for developer-facing products, where the user writes code or integrates a service, are reported by candidates Inferred. Practical consequence: when the user is a developer, the product includes its documentation, errors and first integration, and the first five minutes decide adoption.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Designing before choosing a user	Your first sentence describes a feature	State a segment and a criterion first
2	A demographic user	"Busy parents," "seniors," "students"	Segment by situation and need
3	A long, unranked list of problems	You cannot say which one the solutions address	Rank by frequency, severity and workaround; pick one or two
4	A vague pain point	You could not watch it happen	Write it as something observable
5	Solutions that are variants of one idea	They differ only in size or polish	Offer different kinds: eliminate, automate, assist, reframe, connect

#	Mistake	How to detect it	Fix
6	Technology first	"Use AI to..." before naming the problem	State the problem, then say why the technology is the right tool and what happens when it fails
7	Describing screens	You are talking about buttons and layouts	Describe moments: what the user does, the product does, the user feels
8	No decision	You end on "there are tradeoffs"	Recommend one option and name the sacrifice
9	A generic metric	"Daily active users" for a feature built to solve one problem	Tie the primary metric to the chosen problem, with a guardrail and a counter metric
10	Ignoring misuse, privacy and accessibility	Your risks are all about adoption	Name two or three harms for this product and a design response for each

9. Mental models applied

Four models, each tied to a decision in this chapter. `[[inversion]]` and `[[second-order-effect]]` come from C02, and extreme users from C06.

Inversion (MM-inversion, see `[[inversion]]`, C02). Ask how you would design the product to fail its user: choose a user nobody can picture, solve a problem that is easy to describe but rare, use a technology that works only in the lab, ignore what the device cannot sense. The answers are the checklist for a good design. It is especially useful in the risks step: "How would someone use this against another person?" is inversion applied to a feature.

Extreme users (MM-extreme-users, from C06). Designing for people at the edges of a segment exposes needs that typical users have more mildly. A traveler in a wheelchair finds the step-free route that everyone with a stroller or heavy bags also needs; a person who is deaf needs an alarm that does not depend on sound, and a heavy sleeper benefits from it too. Use the extreme user as a source of ideas and constraints, and test them with typical users before deciding.

The curb-cut effect (MM-curb-cut). A design made for a constraint often benefits everyone. Ramps cut into curbs for wheelchair users also serve people with strollers, suitcases and bicycles. In an interview this is a way to justify an accessible default as a design strength and not a cost: captions help people in noisy places, large targets help people walking, and voice control helps people with wet hands.

Defaults decide behavior (MM-defaults). Defaults have a large effect on what people end up with, so for many users the default is the product [V: Johnson and Goldstein, "Do Defaults Save Lives?", Science 302, 2003: far higher effective consent to organ donation where the default is opt-out; these are consent rates, not donations]. In a design answer, the default is where the privacy, safety and wellbeing choices become real: sharing is off until the person opts in, alerts are quiet until they ask for more, a seated mode is easy to find. Say what the default is, and who it protects. A default that favors engagement over the person is a design choice, and an interviewer can ask you to defend it.

Engineering, design and data-science perspectives

- **Engineering.** Ask early what the device can sense, what can be computed on the device, and what cannot be changed after release. A hardware choice made in week two can fix a constraint for years; a software choice can be patched. Bring an engineer into the problem step, not only the solution step, because they will often know a cheaper way to reach the user's goal.
- **Design and research.** A designer will turn a moment into a flow and show you where it breaks. Ask for the three screens that must be perfect and the ones that can be rough, and ask a researcher what evidence would change the ranking of the pain points. A design review is a place to defend the problem, not only the solution.
- **Data science.** Ask what data exists to size the pain points, and what you would need to instrument to measure the primary metric (C05). Tell the data scientist the decision the metric supports, and agree on the counter metric before launch, so that a good-looking number does not decide by default. Measurement design for the experiment itself is covered in C14.

Checklist: before you say "I recommend"

1. Did I choose a specific user by a stated criterion, and say what I gave up?
2. Is the problem written as something observable, ranked against others, with a reason?
3. Do my solutions differ in kind, and does each link back to the problem?
4. Did I name the deciding criterion before comparing the options?
5. Is the first version described as moments, with what comes next in order?
6. Does the primary metric match the problem, with a guardrail and a counter metric?
7. Did I name two or three harms for this product, each with a design response and a default?
8. Did I say what would change my mind?

10. Case studies

CS-C07-01 · The bike-share first ride · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A bike-share company in a mid-sized city finds that 38% of new riders never take a second ride within 30 days (invented). The operations team believes the cause is bike availability and wants to buy more bikes. The product team is asked to design a better first-ride experience for new riders before the company commits budget.

Constraints. A fixed budget for the quarter, an app team of four, and docks and bikes that cannot be redesigned this year. Any change must work on the existing docks and on a phone with a weak signal.

Data. App logs show that 31% of first-time release attempts fail on the first try (invented). The team watches 12 first rides at three docks and finds that nine riders spend more than two minutes at the dock working out how to release the bike or adjust the seat, and five give up and walk away (invented). A short survey of 400 lapsed riders shows 22% citing "no bikes available" and 41% citing "confusing or stressful the first time" (invented). Several watch the in-app clock run while they struggle.

Problem definition. The belief was that new riders leave because bikes are not available. The evidence points to the first two minutes at the dock: new riders face an unfamiliar release step with the price clock running, and some give up before riding. Written as an observable pain point: "A first-time rider at a dock cannot tell how to release the bike, sees the fare clock running, and abandons the ride."

Analysis. Availability matters to 22% of lapsed riders, and the observed behavior at the dock is stronger evidence than the survey, because it shows what riders do and not what they remember. The riders who struggled described a worry that "the clock is already running," which makes a small confusion expensive. They have workarounds (asking a passer-by, calling support) that are signs of real pain. The pain differs from the operations team's belief in a way that shifts the solution from buying hardware to guiding a first-time moment. The riskiest assumption is that new riders would complete the release if they were guided and not charged while learning.

Options.

Option	Optimizes	Costs
A. Buy more bikes	Availability, the operations team's belief	A large share of the quarter's budget; does not touch the first-ride problem
B. A first-ride mode in the app: a two-step release guide with a short animation, and the fare clock starts when the bike is released, not when the rider scans	The first two minutes	About three weeks of app work; a small revenue change from the free minutes
C. A sticker with the steps on each dock	Visible help for people who do not open the app	Printing and fitting; no guidance for the clock worry
D. B and C together	Coverage	More cost; blurs what worked

Tradeoffs. B directly addresses the observed cause and costs a few weeks. It gives up the possibility that availability matters in some zones, which can be tested later. C is cheap and works without the app, so it helps people who do not have it open, but it cannot fix the clock worry. A is the most expensive and the least connected to the evidence.

Decision. Build B for new riders only, and add C at three docks as a cheap comparison. Primary metric: the share of new riders who complete a second ride within 14 days. Input: the first-attempt release success rate. [[guardrail-metric]]: the safety incident rate for first rides (a guided rider might rush the seat adjustment on a curb), and the revenue per ride for new riders. Counter metric: the rate of new riders who ride once, with the free minutes, and never again.

Outcome. In this fictional scenario the first-attempt release success rate rose from 69% to 88% (invented), and the second-ride rate within 14 days rose from 62% to 71% (invented). The three docks with stickers and the app guide did slightly better than those with the guide alone, a gap too small to treat as a finding (invented). The company bought a small batch of bikes for two zones where availability complaints clustered, and a later test would show whether that mattered.

Alternative outcomes. If the second-ride rate had not moved while release success rose, the team would have returned to interviews, because the dock was not the only barrier. If availability complaints had been concentrated in a segment such as commuters at peak hours, a separate experiment would have been justified. If safety incidents had risen, the free minutes would have been kept and the seat-adjustment step slowed. The lesson is the method: when the owner of a problem names a cause that costs money, watch the first moment of use before agreeing.

CS-C07-02 · Apple's first iPod, 2001: a device, a software link and a price · Verified · Advanced

Sources (types): Apple's press release announcing the product on 23 October 2001 and its press release of 17 July 2002 on the next generation, both read in full (primary, the company's own account); Apple's annual reports on Form 10-K for fiscal 2003 and 2004 (SEC), for unit sales; contemporaneous coverage (Macworld's 2001 report citing an IDC analyst, The Register's launch report, and a CNET piece as reproduced by Mac History); and reference histories for the March 2002 model. The analysis is the author's. This is a design analysis from public information and does not claim knowledge of Apple's internal process. Re-read in October 2026.

Situation. On 23 October 2001 Apple announced the iPod, a portable music player, and said it would ship on 10 November 2001 at \$399. Apple's release described a 5 GB hard drive, a capacity of about a thousand songs, a transfer over FireWire (for which it reported that a CD could be moved in under ten seconds), battery life of up to ten hours, and a weight of about 6.5 ounces. Its tagline, "1,000 songs in your pocket," made the capacity the headline. The player worked with Apple's iTunes 2 software on the Mac through an Auto-Sync feature, Apple's release says that iPod requires iTunes 2, and it described the FireWire transfer as 30 times faster than USB-based players and the drive as giving up to 20 minutes of shock protection [V: Apple press release, 23 October 2001].

Constraints. As documented or reported: the product launched for Mac users, and the software that managed the music at launch ran on the Mac (Apple's release describes syncing to a Mac and requires iTunes 2) [V: Apple press release]; contemporary coverage noted that it did not yet work with Windows [V: Macworld, 2001], and a Windows-compatible generation came on 17 July 2002, shipping with MUSICMATCH Jukebox [V: Apple press release, 17 July 2002]; reference histories say Windows owners had used third-party sync utilities in the meantime [V: reference history]. The player was priced above the nearest competitor with a portable hard drive, which launch coverage put at \$249.99 for the 6 GB Creative Nomad Jukebox (The Register gave around \$220), and a Technology Business Research analyst called the \$399 price "a little high" [V: Macworld, 2001; The Register,

2001; CNET as reproduced]. An IDC analyst said Apple might take some heat for entering consumer electronics, a business with lower margins than its computers [V: CNET as reproduced].

Data. From the release: the specifications above. From launch-week commentary: analysts noted the player's size, battery life and skip protection as advantages, and the FireWire connection as a reason that loading songs was faster than with the USB connections then common. Apple's annual reports on Form 10-K show iPod unit sales of 381,000 in fiscal 2002 (to September 2002), 939,000 in fiscal 2003 and 4.4 million in fiscal 2004, about 5.7 million since the line began, with iPod net sales of \$143 million in fiscal 2002 and \$345 million in fiscal 2003; early totals quoted in some retrospectives differ from these filings [V: Apple Form 10-K, fiscal 2003 and 2004].

Problem definition. *The author's reading of the release.* The problem the release addresses can be written as an observable pain point for a music listener who wants to carry a large collection: the player holds only a few songs, loading songs is slow, and the battery runs out. Apple's release speaks to each: capacity in the tagline, transfer speed in the FireWire claim, and battery life in the specification.

Analysis. *The author's analysis.* Read through this chapter's frameworks, the launch is a design in which several choices serve one problem. Capacity (the tagline) addresses "I can carry only a few songs." The FireWire transfer addresses the slow step that sits between the user's collection and the device, and the software link to iTunes addresses the effort of managing the library, so the user's task is covered from the computer to the pocket. The size came from the choice of a small 1.8-inch drive, which the sources report. This is the hardware-software integration lens (C07.10): the value comes from the three parts working together, not from any one specification. It also shows the surface lens: a device with a few buttons and a small screen needs a way to move around a library of a thousand songs, and Apple's release emphasized the scroll-wheel, which could be operated with one hand and accelerated automatically through long lists, to reach playlists, artists or songs [V: Apple press release].

The costs were also visible at the time. The price was high, and the Mac-only start limited reach to a small share of computer owners, which is a deliberate or accepted narrowing that a product designer would call a choice of first segment (the sources read here do not give the Mac's share) [inference]. The chapter's lesson on defining success applies in reverse: the launch's own claims (capacity, speed, battery life) are the measures of the problem it set out to solve.

Options. *The author's reconstruction of choices a team could have faced; not a record of Apple's deliberations.*

Option	Optimizes	Costs
A. A cheaper flash-based player with room for few songs	Price and size	Capacity, the pain at the center of the problem
B. A hard-drive player with fast transfer and a software link, at a premium price	The whole task from library to pocket	Price; a narrower first audience; hardware that cannot be changed after shipping

Option	Optimizes	Costs
C. A hard-drive player that works with all computers at launch	Reach	More software work and a slower launch; a weaker link between device and software

Tradeoffs. B gives up price and early reach to deliver the full experience, and it accepts the hardware risk that a device, once shipped, is difficult to correct. The software side was updatable; the release said the firmware was upgradable, which it linked to support for future audio formats [V: Apple press release], which is the software-patch side of the hardware-software point. A would have been cheaper and simpler and would have left the capacity problem unsolved. C would have widened the audience at the cost of the integrated experience.

Decision. As documented, Apple launched a Mac-first, 5 GB hard-drive player with FireWire transfer and iTunes integration at \$399, shipping on 10 November 2001.

Outcome. Reported: a 10 GB model followed on 20 March 2002 at \$499 [V: reference histories], and on 17 July 2002 Apple announced a next generation in 5, 10 and 20 GB versions for Mac and Windows, with the Windows version shipping with MUSICMATCH Jukebox [V: Apple press release, 17 July 2002]. The sales figures above come from Apple's annual reports. Whether the integrated design, the timing, the later growth of a music store, or the company's marketing was the cause of the success is a different question, and this case does not settle it.

Alternative outcomes. *Inference.* Had the first audience been too small to reach a self-sustaining volume, the Mac-first choice would have looked like a mistake; the later Windows-compatible generation suggests the audience limit was a real constraint that the company chose to remove. Had capacity not been the dominant pain for listeners, the tagline would not have carried the product. A reader should also note the survivorship problem: a launch that worked is easy to explain afterward, and the same choices in a failed launch would have been read as errors. The method to carry away is the chain: name the pain, address each part of the task from the library to the pocket, state the cost of each choice, and write down what you are giving up.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative**; every figure is an assumption. Each is written against the five behaviors in section 1 and uses the spine in FW-C07-01. The answers are longer than what you would say in the first pass of a real round; they show the reasoning at each step, and a spoken version keeps the same order with fewer words.

IQ-C07-01 · Beginner · product design · PS · Apple

Prompt. Design a feature for a smartwatch aimed at people training for their first marathon.

Rubric.

- **Baseline:** Lists features (heart-rate zones, a training plan, music) with no user choice, no problem and no decision.
- **Strong:** Chooses a specific user, ranks pain points, offers solutions of different kinds, recommends one with a stated cost, and gives a metric.
- **Standout:** Uses the watch's strengths (sensing, haptics, glanceability), addresses injury and wellbeing risk, keeps data on the device, and says what would change the recommendation.

Model answer. I'll assume this is a feature in the watch's existing fitness software, paired with a phone, and that the goal is regular, safe use of the watch during training. Goal ladder: the company wants the watch worn and used every day; the product goal is more first-time marathon trainers who complete their plan; the runner's goal is to finish a first marathon without getting hurt. One tension: pushing runners to train harder may raise use and raise injury risk, so I'll weigh safety above engagement.

User. Four segments: first-time marathon trainers, experienced racers, casual runners and run-walkers. I choose first-time trainers. Their pain is most acute (a long plan with no experience to fall back on), they have the least support, and a watch is the tool they carry on every run. I give up experienced racers, who are a larger and better-served group. *Persona:* Maya follows a 16-week plan, runs before work, and checks her wrist mid-run (invented).

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
Going out too fast in the first third of a long run and fading	Every long run, weekly	High (a bad long run costs confidence)	Checks pace constantly; runs with a friend	1
Not knowing whether to push through soreness or rest	Weekly in the heavy weeks	High (injury)	Searches online; asks friends	2
A fixed plan that breaks when life interrupts	Several times in 16 weeks	Medium	Skips and loses the thread	3
Fueling and hydration timing on long runs	Every long run	Medium	Guesses	4
Motivation sagging in the middle weeks	Weeks 6 to 12	Medium	Quits or repeats the plan	5

I choose the first two as one theme, effort control, because both come from a beginner who cannot yet feel her own limits. I'd confirm the ranking by asking ten recent first-time finishers about their worst long run and their worst week, and I'd expect pacing to rank above fueling.

Solutions that differ in kind. Eliminate: the plan sets the pace for each mile, so the runner never decides. Automate: the watch adjusts the target from heart rate and fatigue signals in real time. Assist: a tap on the wrist when pace is 10% faster than plan, with a different

pattern for too slow. Reframe: train by effort level ("conversational pace") and not by pace, since pace is a poor guide for a beginner on hills and in heat. Connect: a friend or coach pacing remotely.

Evaluation. Eliminate is simple and rigid. Automate is the most powerful, needs reliable signals and can be wrong, and a wrong target costs a beginner a run. Assist is cheap and uses the watch's strength, which is the wrist. Reframe addresses the root of beginners' trouble. Connect depends on other people. I recommend Assist with the Reframe: haptic pacing by effort, for long runs. It gives the runner a calm, glanceless signal at the moment she is drifting, by giving up adaptive plans in the first version. Next I'd add a mid-run effort check, and then plan adjustment after a missed week.

First version as moments. Before the run, the plan sets a target effort range for the day. Mile three: if her effort is above the range for two minutes, the watch gives a slow double tap. If she settles, a single quiet tap confirms it. Mile twelve: if she drifts below, a different pattern. After the run, a one-line summary: "You held your range for 82% of the run" (invented).

Metrics. Primary: the share of planned long runs completed within the target effort range for at least 70% of the run (invented threshold). Input: the share of haptic alerts followed by a return to range within two minutes. Guardrails: the share of runners who switch alerts off within two weeks, and battery use over a long run. Counter metric: the share of runs cut short, since a runner might stop to satisfy the range and not the body.

Risks. Safety: the watch is not a medical device and must never tell a runner to push through pain; any effort advice is capped, and a high-effort signal suggests easing off. Wellbeing: no shame in the summary. Privacy: heart-rate and effort data are processed on the watch and kept on the user's devices by default, with sharing only on opt-in. I'd change my mind if first-time finishers said their problem was fueling and not pacing, in which case the first version becomes a timing assistant.

Company note. Apple is the setting only: Inferred from third-party reports that design prompts at Apple often involve its own devices and weigh privacy and fit with the ecosystem [unverified: candidate-reported]. No Apple data or product behavior is described or implied.

Follow-ups.

- **Q.** How would you cut this down to a version you could build in one quarter? **A.** "I would keep one idea that uses the watch's strengths: plan-aware pacing cues by haptic tap during a run, and a weekly training-load summary using sensors the watch already has. I would leave out social features, nutrition and gear recommendations. The measure is the share of first-time trainers who complete their training plan, with injury-related stops reported by users as the guardrail. A quarter is enough to build, test with a beta group of a few hundred runners over a four-week block, and read plan completion against a comparison group."
- **Q.** How do you make sure the feature does not push runners to overtrain? **A.** "By building the limit into the design. The plan should cap the week-over-week increase in load at a conservative level set with a sports-medicine adviser, include scheduled rest days, and prompt a lighter day when recovery signals are poor. I would avoid streak mechanics that

reward training through fatigue. The success metric is completion of the plan, not total distance, and the guardrails are self-reported pain or injury and the share of users who stop after a prompted rest day. If injury reports rise in the beta, I would pause the roll-out and review the load rules."

IQ-C07-02 · Beginner · product design · PS

Prompt. Design an alarm clock for people who are deaf.

Rubric.

- **Baseline:** "Use flashing lights and vibration," with no user detail, no problem and no recommendation.
- **Strong:** Notes that the user is given, spends time on the problem and its failure cost, offers different kinds of solutions, and recommends one.
- **Standout:** Treats reliability as the central requirement, designs a backup, considers the sleeping partner and travel, and reframes the design as one that benefits everyone.

Model answer. The user is given, so I'll skip segmentation, confirm the user, and spend the time on the problem. "Deaf" covers a range: people who are deaf, hard of hearing and use hearing aids (which they usually remove at night), and people who use sign language as a first language. The common fact is that a sound alarm will not wake them. I'd confirm that with a quick check of how they wake up today.

Goal. The company wants a physical product that people trust and recommend; the user's goal is to wake on time, every time, without help from another person.

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
A cue strong enough to wake from deep sleep	Every night	Very high (a missed morning costs work or a flight)	Asks a partner; sleeps lightly; uses several alarms	1
Trusting the alarm: a dead battery or a failed alarm is silent	Every night, as a worry	High	Checks the alarm repeatedly	2
Waking a partner who can hear	Every morning if there is one	Medium	Sets the cue on a wristband	3
Travel: hotel rooms offer no equivalent	Per trip	High	Brings a device; asks the front desk	4
The body adjusts to one cue and stops responding	Over weeks	Medium	Changes the alarm	5

I choose the first two. The central requirement of an alarm is reliability, so the design must make failure visible before sleep.

Solutions that differ in kind. Assist: a wristband that vibrates at the set time. Automate: a bed shaker, which is stronger, that starts weakly and builds. Reframe: a lamp that brightens over 20 minutes and works with the body's waking cycle, so that the alarm is a gradual cue. Connect: a call or message to a friend or relative as a last resort. Eliminate: a combination that removes the worry itself, by checking its own state every night.

Recommendation. I recommend a wristband as the primary cue, with a bed shaker as backup and the phone as a last resort, in the first version. This gives up elegance (three parts) to buy reliability. Before sleep, the wristband shows a green light for "armed and charged," and a red light if anything is wrong. Next I'd add the lamp for a gentler wake, and travel pairing so the wristband works from the phone in a hotel. I'd drop the backup only if testing showed the wristband alone woke people more than 99.9% of the time (invented), and I would not drop the green-light check.

Metrics. Primary: the share of alarms acknowledged by touch within three minutes, as a proxy for waking on time. Input: the share of nights the device shows "armed" before the user sleeps. Guardrail: the failure-to-fire rate, which must be near zero and is the metric that matters most; battery-related failures. Counter metric: partner-disturbance reports, so the design does not fix one problem and create another.

Risks. Reliability is the safety risk, and the design answer is the self-check and the redundancy. Skin comfort for a wristband worn all night. Dignity: the product should not be marketed or shaped as a device "for deaf people"; heavy sleepers, shift workers and people sharing a bed with a light sleeper would all want a silent, strong cue, which is the curb-cut effect (section 9). I'd change my mind if users said they already trusted their phone's vibration and wanted software, not hardware.

Follow-ups.

- **Q.** What if the vibrating alarm fails during the night? **A.** "Reliability is the product, so I would design for failure. Two independent wake mechanisms: a vibrating unit under the pillow or mattress and a bright light, plus a connection to the person's phone or watch as a third. A battery warning days before the battery runs out, a test button, and a nightly self-check that tells the user at bedtime if a mechanism is not working. I would define the metric as the missed-wake rate and set the tolerance very low, then test with 20 users for four weeks, with a diary and device logs, before launch. Any miss is investigated, not averaged away."
- **Q.** How would you make it valuable beyond the deaf community without diluting it? **A.** "Many people benefit from a strong, silent alarm: heavy sleepers, shift workers, and couples who wake at different times. The core requirements stay set by the deaf user, because they are the strictest, and I would add features for the others only if they do not reduce reliability. I would measure the share of buyers who are outside the deaf community and satisfaction in both groups. A wider market lowers the unit cost and supports the product, and the guardrail is that the missed-wake rate for the primary users does not worsen."

IQ-C07-03 · Intermediate · product design · PS · Google

Prompt. Design a feature in a maps app for people who use wheelchairs.

Rubric.

- **Baseline:** "Show wheelchair-accessible routes," with no analysis of data, failure or segments.
- **Strong:** Identifies what makes a route accessible, ranks the pain points, addresses where the data comes from, and recommends a version.
- **Standout:** Sees the failure cost (being stranded), shows confidence rather than false certainty, uses community and partner data with safeguards, and avoids inferring disability.

Model answer. I'll assume a feature in an existing maps app on phones, with the goal that people trust the app for everyday trips. The user is given as people who use wheelchairs, but it spans manual and powered chairs, and people with other step-free needs (strollers, walking limits, heavy luggage) share part of the need. I'll design for wheelchair users first, as the extreme user whose needs are strict: one step or one steep slope blocks the route.

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
A route that looks fine on the map has a step, a steep ramp or construction	Weekly for active users (invented)	Very high (stranded far from home)	Scouts in advance; avoids new places	1
Missing information: many places have no step-free data	Most new trips	High	Calls the venue	2
The accessible entrance is not the main entrance	Common at older buildings	Medium	Searches for the side entrance	3
Lift outages on transit	Occasional	High	Checks the operator's site	4
Routes that are technically step-free but much longer	Often	Medium	Accepts it	5

I choose the first two. They are one problem: the app cannot yet tell the user how much to trust a route. Wrong information is worse than none, because a confident wrong route strands a person.

Solutions that differ in kind. Automate: infer step-free paths from street imagery and sensor data. Assist: step-free routing that labels each route with how sure the app is and where the information came from. Reframe: plan to the accessible entrance, not the street address. Connect: let users and venues confirm and correct data. Eliminate: not possible for the app alone; I'd recommend that venues and cities publish the data.

Recommendation. I recommend Assist with Connect for the first version: step-free routing with confidence labels, in a few areas where data is good, with a one-tap way to report a blocker. It gives users an honest route by giving up broad coverage at launch. Next I'd add the accessible-entrance guidance and lift status; then automated inference to widen coverage, with human confirmation for what it adds. I'd bring the inference forward if reports showed the data gaps were the dominant failure.

First version as moments. The user turns on "step-free" in settings once. Each route shows "Confirmed step-free" (a recent user or venue confirmation), "Likely step-free" (inferred), or "Unknown on this segment," which the user can inspect on a map. During the trip, one tap reports "blocked here." The report corrects the data and warns the next user within minutes.

Metrics. Primary: the share of step-free routes completed without a reported blocker. Input: the share of route length with confirmed data in the launch area. Guardrails: the blocker-report rate per 1,000 routes, and the time from a report to a corrected route. Counter metric: abandonment of routes labeled "unknown," which tells us whether honest uncertainty drives people away or whether they accept it.

Risks. False assurance: a "confirmed" label must expire and be re-checked, and the app must not show certainty it lacks. Data abuse: crowd contributions can be wrong or malicious, so changes need corroboration. Privacy: the app must not infer that someone has a disability from how they move, and the setting is a choice, not a guess; contribution is opt-in, and location trails are not stored for it. I'd change my mind if wheelchair users said that they wanted the app mainly for the venues' entrance information, in which case the first version is a venue layer and not routing.

Company note. Google is the setting only: `Inferred` from candidate reports that product-design prompts at Google can concern its own products such as maps, often at global scale [unverified: `candidate-reported`]. No Google product behavior is described or implied.

Follow-ups.

- **Q.** How do you handle route data that may be wrong or out of date? **A.** "By showing confidence and source, not false certainty. Each segment would carry how it was verified (a partner, a community report, or inference from map data) and when. Reports expire if not reconfirmed, say after twelve months, and routing would prefer verified segments and flag uncertain ones. Users can report a barrier in two taps, and fixes reach other users quickly. I would audit accuracy by sampling routes in a few cities and comparing the app's claim with a physical check, and track the share of routes where a user had to turn back as the primary quality measure."
- **Q.** How do you avoid inferring that someone has a disability? **A.** "I would not infer it at all. Step-free routing would be an explicit setting that the user chooses, stored with their preferences and not derived from their movement. I would minimize the data kept, keep the preference out of any advertising use, and have privacy and legal review the design. The setting should be useful to anyone who needs it (a parent with a stroller, a traveler with heavy luggage), which also reduces stigma. A guardrail is that no part of the product reveals or uses the setting for any purpose other than routing."

IQ-C07-04 · Intermediate · product design · PS

Prompt. Design a feature to help people who join a meeting remotely feel included when others are together in a room.

Rubric.

- **Baseline:** "Better cameras and a chat window," with no analysis of the problem.

- **Strong:** Identifies the remote participant's specific problems, ranks them, offers different kinds of solutions and measures inclusion.
- **Standout:** Chooses a metric that captures equity without forcing participation, protects people in the room, and considers the organizer's role.

Model answer. I'll assume a feature in an existing video-meeting product, used at work, on a laptop for remote people and a shared screen and camera in the room. The goal: remote participation that is as effective as in-room. Goal ladder: the company wants the product to be the default for hybrid work; the product goal is balanced participation; the remote person's goal is to be heard and to follow what is happening.

User. Four roles: the remote participant, the in-room participant, the meeting organizer and the person who sets up the room. I choose the remote participant, because their disadvantage is structural and they cannot fix it themselves. I'll note the organizer as the person who can change the meeting's behavior, since a feature that depends on the organizer's discipline will fail.

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
Cannot jump into the conversation: the room speaks over the gaps	Most hybrid meetings	High	Waits; types in chat; stays quiet	1
Cannot hear side conversations or the quiet speaker in the room	Often	High	Asks "can you repeat that?"	2
Cannot see the whiteboard or who is speaking	Often	Medium	Asks someone to share a photo	3
Chat is missed by the room	Often	Medium	Hopes a person in the room reads it	4
Feels invisible after the meeting: decisions made in the corridor	Occasional	Medium	Follows up in messages	5

I choose the first two. They are the heart of exclusion: the remote person cannot enter and cannot hear.

Solutions that differ in kind. Eliminate: a "remote-first" mode where everyone joins on their own device, with room speakers muted, so nobody has an advantage. Automate: a camera that frames and follows the active speaker, with audio that favors the quietest voices. Assist: a visible queue so remote people can raise a hand and the room sees a light or an on-screen line. Reframe: a rotating "remote advocate" role in the room. Connect: a post-meeting summary of decisions for everyone.

Recommendation. I recommend the queue with speaker framing for the first version. It gives remote people a way into the conversation without depending on anyone's goodwill by giving up the full fairness of remote-first, which needs a changed habit. Next I'd add the post-meeting decision summary, and offer remote-first as a mode organizers can choose. I'd bring remote-first forward if organizers adopted it readily.

Metrics. Primary: the ratio of remote participants' share of speaking turns to their share of attendees, with a target near one. Input: the share of raised-hand requests acknowledged within 30 seconds. Guardrails: meeting length, and in-room participants' satisfaction, so that the feature does not slow meetings down or annoy the room. Counter metric: forced turns, such as a remote person being called on when they did not raise a hand, since the aim is the option to speak, not a quota.

Risks. Privacy: room cameras and microphones capture people who did not join the call, so the room device shows a clear recording indicator and a consent step. Wellbeing: a speaking-time metric must not be turned into an individual score for performance reviews, so it is shown in aggregate to the organizer only. Accessibility: captions on by default for the remote side, and the queue works by keyboard and voice. I'd change my mind if the data showed that remote people's participation was already balanced and the problem was information, not speaking.

Follow-ups.

- **Q.** How would you measure inclusion without forcing people to speak? **A.** "With meeting-level measures that are not surveillance. Compare the speaking-time share of remote participants with their share of attendees, the time between a remote person raising a hand and being called on, and a one-question pulse at the end of a meeting: I could follow and contribute. I would aggregate at the meeting or team level and not show scores for individuals to managers, because individual scoring can chill participation. The guardrails are meeting length and the complaints of people in the room about disruption."
- **Q.** What would you ask the organizer to do, and what if they do not? **A.** "Little, and I would make the right behavior the default. A visible raise-hand queue on the room screen and a short prompt to the organizer at the start of the meeting to take remote questions first would be on by default. If organizers ignore prompts, the default does the work: the queue shows remote hands whether or not the organizer mentions it. I would measure how often the queue is used and the call-on delay, and adjust if the default annoys people in the room."

IQ-C07-05 · Hard · product design · PS · Apple

Prompt. Design a way for family members to share location with strong privacy controls.

Rubric.

- **Baseline:** "A live map of the family with a privacy toggle," with no analysis of misuse or consent.
- **Strong:** Identifies the real need, builds consent into the design, covers vulnerable users (children and people in controlling relationships) and sets protective defaults.
- **Standout:** Shares the minimum that serves the need (events and time limits before a constant map), protects the person who pauses, sets a launch condition or kill criterion, and recognizes that a feature can be a tool of control.

Model answer. I'll design this from first principles, not describe any existing product. I'll assume a feature on phones and watches that a family can turn on, with a goal of helping families coordinate and feel safe. Goal ladder: the company wants a family to find the platform useful and trustworthy; the product goal is that the feature makes family life easier without a loss of trust; each person's goal differs, and that is the central difficulty. A parent wants to know a child got home. A teenager wants privacy. An older parent wants help without losing independence. In an abusive relationship, location sharing can be a tool of control. I'll put the responsible-design check (FW-C07-06) first, because it shapes the design, not just the end of the answer.

User. By situation: parents of younger children (about 8 to 12), parents of teenagers, adults checking on an aging parent, and partners. I choose parents of children about 8 to 12 as the first group. The need is clear (arrival, pickups), the child has little reason to hide, and the family structure makes consent simpler. I give up partners and teenagers for the first version, since the misuse and consent issues are hardest there, and I'd treat them as separate designs and not as a toggle on the first.

Pain points (family coordination; I'd verify with interviews).

Pain point	Frequency	Severity	Workaround today	Rank
"Are you there yet?" messages around travel	Daily	Low to medium	Texts and calls	1
Worry between a child leaving and arriving	Daily	Medium to high	Waits for a call	2
Pickup coordination	Several times a week	Medium	Calls and waits	3
A person cannot tell who can see their location	If sharing exists, always	High (trust)	Turns it off	4
Sharing used to control another person	Rare, severe	Very high	None for the person affected	5

The first three show what the need really is: arrival, not constant tracking. I choose to design around events and trips, and treat the last two as design requirements, not ranked problems, because their severity is too high to trade against convenience.

Solutions that differ in kind. Eliminate: no location sharing; an "arrived" message only. Automate: automatic sharing along a regular route. Assist: sharing for a set trip or time, which ends by itself. Reframe: share the arrival time (an ETA) and not the position. Connect: a shared place list ("home," "school") with arrival and departure notices.

Recommendation. I recommend the combination of Reframe and Assist in the first version: arrival and departure notices for places the family agrees on, plus time-limited sharing for a single trip, with no always-on map by default. It gives families the thing they asked for with the least exposure, by giving up the simplicity of a constant live view. Next I'd add an optional continuous view for families who choose it, with the controls below; and later, versions for teenagers and for adults and aging parents, each with its own consent flow.

The design, by the checks. Consent: nothing is shared until each person accepts, and each person sees the same thing the others see about them. *Visibility:* a persistent indicator shows when sharing is active and with whom. *Control:* a one-tap pause or stop for the person being shared, and sharing for trips ends automatically. *Protection for the person who pauses:* others see "location unavailable," which looks the same whether sharing is paused, the phone is off or there is no signal, so that pausing does not expose someone in an unsafe situation, and the design discusses openly the cost, which is that a parent may worry. *Children:* age-based defaults, with a child's consent step written in plain words, and the parent's controls stop at the minimum needed. *Data:* the minimum location history, stored on the user's devices where possible, with a short retention period, and not used for advertising. *Misuse:* an easy way to exit a sharing relationship and to find out who has been sharing with you, and a route to support for someone being tracked without consent.

Metrics. Primary: the share of sharing relationships still active after 90 days in which both people report that the arrangement is comfortable, from a short survey. Input: the share of relationships set up with places and trips rather than continuous sharing. Guardrails: abuse and harassment reports per 10,000 relationships, and the time to resolve each. Counter metric: the share of relationships ended in the first week, which could mean friction is too high, or could mean the feature surfaced a conflict that needed a conversation.

Launch condition. I would not launch if any path allowed sharing to begin without the person's knowledge, or if testing with safety experts found a way to use the feature covertly that we could not close. I'd also hold the launch if the abuse-report rate in a limited release exceeded a threshold set in advance. I'd change my recommendation if research showed that families wanted a constant view badly enough that they set it up anyway on a worse product; then the better course is a well-designed continuous option with strong defaults. Rules about children's data and consent differ by country [V: rules differ by country, for example India's DPDP Act treats anyone under 18 as a child], so I'd involve legal and safety teams from the start.

Company note. Apple is the setting only: Public that Apple publishes a privacy position and Inferred from third-party reports that its design prompts weigh privacy and safety [V: Apple privacy pages; the prompt descriptions are candidate-reported and unverified]. This answer does not describe how any existing product works.

Follow-ups.

- **Q.** How do you protect someone for whom location sharing could be dangerous? **A.** "By making sharing visible, limited and easy to end. Sharing starts only by invitation that the person accepts; each person can see who can see them; sharing can be time-limited with periodic reminders; and ending it is one tap. Whether the other person is told that sharing stopped is a safety decision I would make with domestic-violence and child-safety experts, because a notice can itself create danger. I would offer event-based sharing (arrived home) before a constant map. The guardrail is abuse reports per 100,000 users, and I would treat a rise as a reason to pause."
- **Q.** What launch condition or kill criterion would you set? **A.** "Launch conditions: review by outside safety experts, a staged roll-out starting with a small share of users in one country, and a clear process for abuse reports. Kill or pause criteria written in advance: if

confirmed misuse reports exceed a stated rate (for example 2 per 100,000 users in a month), or if experts identify a design flaw that cannot be fixed quickly, pause new sharing. Success is not engagement: I would measure opt-in rate, the share who end sharing within a week, and satisfaction of both parties, and accept that a feature used less may be a safer one."

IQ-C07-06 · Hard · product design · PS

Prompt. Design a smart pill bottle for older adults who take many medications.

Rubric.

- **Baseline:** "A bottle that reminds you to take your pills and tells your family if you miss one."
- **Strong:** Identifies the real problems, considers non-device alternatives, offers different kinds of solutions and sees the safety stakes.
- **Standout:** Notes what the device can and cannot know (opening is not swallowing), plans for failure and battery, treats hardware as hard to change, and tests before building.

Model answer. I'll assume a hardware-and-app product, sold to older adults or to their adult children, with a goal of taking medications as prescribed. The company's goal is a product that people and doctors trust; the user's goal is to stay well and independent. Because it is hardware that touches health, I'll put reliability and safety in the first minutes.

User. Segments by situation: older adults managing five or more daily medications on their own, those whose adult child helps, and those with a paid caregiver. I choose people managing on their own with a family member who worries, because the pain is daily, the stakes are health, and the family is the buyer. I give up the caregiver market, which has different buyers and rules. There are two users: the person who takes the pills, and the person who worries.

Pain points.

Pain point	Frequency	Severity	Workaround today	Rank
Forgetting a dose, or doubting whether one was taken	Several times a week	High	Pillbox with days; a note	1
Taking a dose twice	Occasional	Very high	Checks the pillbox	2
Confusing similar pills or several bottles	Daily	High	Labels, memory	3
Running out of a medication	Monthly	High	Calendar reminders	4
A family member who cannot tell how things are going	Weekly	Medium	Calls	5

I choose the first two, which share a cause: no reliable record of what was taken, and no gentle cue.

Solutions that differ in kind. Eliminate: the pharmacy prepares packs sorted by day and time, so there is nothing to sort and nothing to confuse; this needs no electronics. Automate: a dispenser that releases the right pills at the right time. Assist: a smart cap with a light and chime that records when the bottle is opened. Reframe: tie doses to a daily routine, such as meals or brushing teeth, with a cue placed where the routine happens. Connect: an alert to a family member when a dose looks missed.

Evaluation and recommendation. The strongest first move is often the cheapest one, and I'd say so: the pharmacy packs solve confusion without any new hardware. For the device, the assist-style cap is cheaper and simpler than a dispenser, which is expensive, bulky and risky if it releases the wrong pills. I recommend a combined first version: the cap with light and chime, working with pharmacy packs where the pharmacy supports them, and a gentle, optional family summary. It gives a cue and a record by giving up the dispenser's automation. Next I'd add refill reminders, then the dispenser for people with the most complex regimens.

What the device can and cannot know. The cap knows it was opened, not that the pill was swallowed. A "dose taken" label would be a claim the device cannot support, so it says "bottle opened" and the app asks, "Did you take it?" before it tells anyone anything. An opening is a proxy for a dose (a [[proxy-metric]] in C05's terms), and the design must not treat it as the dose.

Failure and hardware. If the battery dies, the bottle must still work as a normal bottle, and a low battery alert goes out days earlier. Because hardware cannot easily be changed after shipping, I'd fix the safety behaviors in software that can be updated, and keep the physical design simple. I'd run a concierge prototype before manufacturing: 20 households (invented) with pre-sorted packs and daily text prompts for four weeks, to learn which cue the person responds to and whether the family summary helps or irritates.

Metrics. Primary: the share of scheduled doses with a bottle opening or a confirmed "taken" within a window set by the prescriber's instructions. Input: the share of users who respond to the cue within 15 minutes. Guardrails: failure-to-alert events (missed alarms from battery or connection faults, which should be close to zero) and double-open events. Counter metric: the rate at which people stop using the cap after the first month, since a reminder that nags gets abandoned.

Risks. Safety: wrong or late alerts could cause harm, so the app never advises on dosing and always sends questions about dosage to the person's clinician or pharmacist. Dignity and consent: the family summary is the older person's choice and shown to them. Privacy: medication records are sensitive health data, so they are stored with the minimum needed and shared only by opt-in. Regulation: a device that makes health claims may fall under medical-device rules that vary by country [V: FDA, EU MDR and India's Medical Devices Rules 2017 each regulate software with a medical purpose], so the claims stay modest. I'd change my mind if the prototype showed that pharmacy packs alone solved the problem, in which case the product is the pack, and the cap is unnecessary.

Follow-ups.

- **Q.** The bottle can tell when it was opened but not whether the pill was swallowed. How do you handle that? **A.** "I would be honest in the design and the claim. The signal is opened, not taken. I would let the user confirm with a tap, show the caregiver only what is known, and avoid alarming a family on a weak signal; a late opening might trigger a gentle reminder to the user before anyone else is told. I would cross-check with refill timing over weeks. The claim in marketing and in the app should say the bottle tracks openings and helps with reminders, not that it ensures adherence, which also keeps the product clear of medical-claim issues."
- **Q.** How would you test the idea before building hardware? **A.** "With a concierge test: give 20 older adults a standard pill organizer plus text reminders, and with their consent a daily message to a family member, for four weeks. Measure self-reported missed doses, the family member's usefulness rating and how often the reminders were ignored. Threshold: at least a 30% reduction in missed doses against their baseline week, and at least 15 of 20 willing to continue. If it passes, build a simple prototype with only opening detection and a reminder. If not, the problem is not the lack of a smart bottle."

IQ-C07-07 · Hard · product design · PS

Prompt. Design a product that helps people reduce food waste at home. (*Midway through your answer, the interviewer says: "I'm not sure your user is the biggest source of waste. Why not restaurants?"*)

Rubric.

- **Baseline:** Gets defensive or abandons the choice at once, and has no criteria to settle the question.
- **Strong:** Treats the challenge as a chance to restate the criteria, defends or revises the choice with reasons, and keeps the spine.
- **Standout:** Separates the size of the problem from the product's right to win, states what evidence would change the choice, answers the challenge in under a minute, and addresses the safety risk of the design.

Model answer. *The first part of the answer, in brief.* I'll assume a consumer app from a company with a grocery or a smart-home reach, and a goal of helping people waste less food and save money. I choose households that cook most weeknight meals and shop weekly, because the waste is regular and they can change it. Pain points, ranked: buying more than they use, forgetting what is in the fridge until it spoils, not knowing whether food is still safe, and cooking portions that are too large. I choose the first two as one problem, buying and storing without a view of what is already there.

The challenge. "That's a fair challenge, and the way to answer it is by the criteria I used for the user: size, pain, reach and fit. On size, UNEP's Food Waste Index Report 2024 puts households as the largest source of food waste in 2022 (60 percent), ahead of food services (28 percent) and retail (12 percent), though the split depends on the country [V: UNEP and WRAP, Food Waste Index Report 2024: 2022 data, households 60 percent, food services 28 percent, retail 12 percent; 1.05 billion tonnes]. If it were the other way, that alone would

not decide it, because restaurants and households differ in the other criteria. A restaurant has a strong money reason to cut waste and a small number of decision-makers, so a B2B tool can reach it directly. A household has a weak reason for any single meal, and many decision-makers, which is why a consumer product is harder but also less served.

Fit decides the rest. I assumed a consumer company with reach into homes, which favors households. If the company were a restaurant-supply business, I'd switch. So my position is that I'd keep households for this prompt, and the evidence that would move me is the company's strengths and a size figure in this country that put restaurants far ahead."

Then the answer continues. Solutions that differ in kind: Eliminate: a shopping list built from what is already in the kitchen, so less is bought. Automate: the fridge or the receipt identifies what is there. Assist: an alert two days before an item's date with a recipe that uses it. Reframe: plan meals around what must be used, "use-first" cooking. Connect: share or swap surplus with neighbors. I recommend the Assist and Eliminate pair for the first version, a use-first list with one nudge per day, because it needs no new hardware and acts at the moment a decision is made, in the kitchen and in the shop. It gives up automatic detection, which depends on receipts or sensors that are hard to get right.

Metrics. Primary: the share of items in the use-first list that are eaten before they spoil, from a quick self-report tap, and where a shop partner exists, the reduction in items bought and thrown. Input: the share of weeks in which the household checks the list before shopping. Guardrails: nudges per week, and opt-outs. Counter metric: food-safety incidents reported, and the share of people who say the app made them feel guilty.

Risks. Safety is the main one, and it is specific to this product: a feature that pushes people to eat food that is spoiled can harm them. The design response is to treat "use by" dates as safety limits and "best before" dates as quality guides, to be conservative in the wording, and never to tell a user that a food is safe; when in doubt it says to throw it away. Wellbeing: no guilt, no shaming counters. Privacy: receipts and shopping lists reveal habits and health, so they stay on the device by default. I'd change my mind if the first tests showed that the problem was not forgetting but over-buying by habit in the shop, which would move the product from the fridge to the shopping list.

Follow-ups.

- **Q.** How would you answer the challenge that restaurants are a bigger source of waste?
A. "In under a minute. Restaurants may be a larger source of waste per site, and the size of a problem is not the same as our right to win. I would say: I chose households because we reach them directly, the behavior is changeable with information, and we can learn within weeks. A restaurant product is a business-to-business sale with different buyers and longer cycles. I would compare shares of waste by source using public reports and our access to each, and I would keep restaurants as a later segment. What would change my choice: a partnership that gives us restaurant access, or evidence that household waste is a small share."
- **Q.** How does the design avoid safety risks, such as telling someone food is safe to eat? **A.** "The product should never declare food safe. It would show quality guidance (use soon, best by) linked to official food-safety guidance, say plainly that dates are a guide and that

when in doubt people should throw food out, and make that the default message for high-risk items such as meat and dairy. I would have food-safety experts review the wording and track user reports of illness as a guardrail. The aim is to reduce waste from forgotten food, not to push people to eat food they are unsure about."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked.

EX-C07-01 · Observable or vague?

Problem. Decide which of these pain points are observable and rewrite the rest so that you could watch them happen: (a) "People find budgeting stressful." (b) "A shopper cannot tell whether an online jacket will fit and orders two sizes." (c) "Parents are busy." (d) "A runner checks her wrist four times in the first mile because she does not trust her pace." (e) "Learning a language is hard." (f) "A new driver stalls at a hill start and the cars behind honk."

Context. You are listing pain points for a design answer and the interviewer will ask you to show one.

Data. None beyond the list.

Constraints. One rewrite per item, no more than 25 words each.

Expected approach. A pain point names a person, a moment and an observable difficulty (FW-C07-03). Items that name a feeling or a trait need a situation.

Solution. (b), (d) and (f) are observable already. Rewrites: (a) "A salaried employee opens her banking app on the last week of the month, sees an unexpected balance and stops opening it until payday." (c) "A parent of two works until 6:00 and cannot make dinner before 7:30, so orders delivery on three weeknights." (e) "A beginner learning Spanish understands a recorded sentence but cannot answer a live one, and stops practicing speaking."

Alternative solution. Test each item with "could I film it?" If not, add the missing person, place or moment.

Tradeoffs. Specific pain points are easier to rank and design for, and they can be too narrow; keep each one tied to a segment so that it does not read as a single anecdote.

EX-C07-02 · Rank the pain points

Problem. Rank these six pain points for people who cook weeknight dinners after work, and choose the top one or two with a reason: (1) deciding what to cook, (2) shopping for missing ingredients, (3) following a recipe with wet hands, (4) cooking for different tastes, (5) cleaning up, (6) food spoiling before use.

Context. A recipe-app team wants a design direction.

Data. Assume from interviews (invented): (1) happens on 4 of 5 weeknights and causes a 15-minute delay or a delivery order; (2) on 2 of 5 nights and adds a trip of 20 minutes; (3) every night, a minor annoyance; (4) on 2 of 5 nights, a source of argument; (5) every night, disliked but accepted; (6) weekly, with a cost of about \$10 a week.

Constraints. Five minutes.

Expected approach. Use frequency, severity and workaround in the pain-point matrix.

Solution. Rank (1) first: highest frequency, a real cost (delay or delivery, which also costs money), and an active workaround (ordering delivery). Rank (2) second: moderate frequency, 20 minutes each time, and tied to (1), since a decision about dinner leads to missing ingredients. Then (4), (6), (3), (5). Choose (1) and (2) as one problem: "no plan that matches what is in the kitchen." State the evidence that would change the ranking: if interviews showed (4) caused most delivery orders, it would move up.

Alternative solution. Combine (1), (2) and (6) as "the gap between what is in the kitchen and what I decide to cook," and name it as the problem, then rank solutions, not pain points.

Tradeoffs. Merging pain points gives a stronger problem and hides how much each contributes; keep the separate ranks visible to the interviewer.

EX-C07-03 · Different in kind

Problem. For the problem "new parents cannot get enough sleep," classify these ideas by kind (eliminate, automate, assist, reframe, connect) and add one idea for any missing kind: (a) a baby monitor that rocks the cot when the baby stirs; (b) an app that tells parents when the baby will probably wake next; (c) a service that arranges night-time help from a vetted volunteer; (d) a way to share night shifts between two parents; (e) a sleep-training program that teaches the baby to settle on their own.

Context. A consumer health company is exploring new-parent products.

Data. None.

Constraints. One line per idea.

Expected approach. Apply the solution spectrum (FW-C07-04) and check that the final set differs in kind.

Solution. (a) Automate. (b) Assist. (c) Connect. (d) Reframe (it changes the parents' goal from "be awake together" to "sleep in blocks"). (e) Eliminate (it removes the cause by changing the baby's waking pattern). All five kinds are present; a safety check is needed for (a) and (e), since a rocking cot and sleep training involve infant safety and must follow safe-sleep guidance.

Alternative solution. If the list had no Eliminate idea, you could add: "a feeding setup that does not need either parent to wake fully," then check it against safety guidance.

Tradeoffs. A wide spread shows range, but ideas that touch infant safety need extra care, and a candidate should say so rather than offer them freely.

EX-C07-04 · Choose a user

Problem. Choose a first user for a product that helps people adopt a pet, scoring at least three segments on size, pain intensity, reach and fit, and name what you give up.

Context. The company runs a marketplace app for local services.

Data. Segments (invented): first-time adopters, families adding a second pet, people who adopt from shelters, and people who adopt older pets. Assume roughly equal sizes except families (smaller).

Constraints. Seven minutes.

Expected approach. Use the segment scorecard (FW-C06-02), with a stated criterion.

Solution. First-time adopters score high on pain (they do not know what to expect, what a pet costs or how to prepare), medium to high on reach (they search for guides and local services, which the marketplace can serve), and high on fit (the company's services, such as vets and pet-sitters, match their next needs). Families adding a second pet have a lower pain, having done it before. Shelter adopters are a channel and not a need. I choose first-time adopters by pain and fit, and give up the loyalty and the lower cost of reaching families that already own pets. Evidence to confirm: interviews with recent adopters about the first two weeks.

Alternative solution. Segment by decision stage (thinking about it, choosing, first month) and choose the first month, when needs for services are highest.

Tradeoffs. Choosing by pain and fit favors a segment that adopts once; the company's lifetime value may sit with those who stay and add services, which a later step can address.

EX-C07-05 · From phone to watch to voice

Problem. A phone app lets people build a grocery list by typing and scanning barcodes. Redesign the core moment (adding an item) for a watch and for a voice-only speaker, and say what each surface makes easier and harder.

Context. The team wants to know whether the feature belongs on other devices.

Data. Assumed: the user adds items mostly when they notice something has run out, often in the kitchen with hands busy.

Constraints. Five minutes.

Expected approach. Use the surface lens (FW-C07-07): attention, input, sensing, battery, privacy, continuity.

Solution. *Watch:* a raised-wrist moment: "Add milk" by dictation, confirmed by a tap. Easier: capture in the moment, with no phone. Harder: long names, duplicates, errors in dictation (the confirmation line matters). *Speaker:* "Add oat milk to the list," a spoken confirmation, and a later read-back by aisle. Easier: hands-free in the kitchen, the exact moment the need arises. Harder: no visual check, a household with several users (whose list?), and privacy at home. The phone stays the place to review the list and shop; the other two are capture surfaces, which is the continuity idea.

Alternative solution. Decide the watch is not worth building, because the speaker already covers the kitchen moment and a watch adds little, and say so.

Tradeoffs. Every added surface costs maintenance and consistency; add one only if it serves a moment the others cannot.

EX-C07-06 · Run the responsible-design check

Problem. A running app proposes "share my live run with friends." Run the check (FW-C07-06) and name the two most serious risks with a design response and a default for each.

Context. The feature shows a map of the runner's route in real time to chosen friends.

Data. None.

Constraints. Five minutes.

Expected approach. Go through the check and pick the two most relevant, then state the response, not only the risk.

Solution. *Misuse and safety:* a live route can reveal the runner's home (the start), regular times and current position to someone who means harm. Response: sharing is off by default, enabled per run, to named people; the first and last stretch are hidden; the runner sees who is watching; a friend can be removed without notification. *Privacy and consent:* the runner's friends should not be shared with others, and a shared link must not be guessable. Response: sharing links expire at the end of the run, and the route is not public or searchable. Also note accessibility (voice control and large targets) and failure (a stale position that shows an old location should show its age).

Alternative solution. Replace live sharing with an end-of-run summary sent after a delay. It gives up the safety benefit some runners want, a companion who can see them, and removes the real-time risk.

Tradeoffs. Hiding the start and end protects the runner and reduces the accuracy of the route; the delay removes risk and some of the feature's purpose. State both.

EX-C07-07 · Fill the grid and write the recommendation

Problem. Three options are proposed for "help new employees feel connected in their first month": (A) a buddy program matched by the manager, (B) an app that suggests colleagues to meet by shared interests, (C) a weekly small-group lunch hosted by someone not in their team. Fill in an evaluation grid and write the recommendation line.

Context. A company of 2,000 people (invented) with offices in three cities; a team of two people can build something.

Data. Assumed: A costs manager time (about 2 hours a month each); B needs a month of engineering and good data on interests; C needs an organizer and a budget of \$15 per person per lunch (invented).

Constraints. Seven minutes.

Expected approach. Use the evaluation grid and recommendation line (FW-C07-05); name the deciding criterion first.

Solution. Deciding criterion: impact on connection in the first month for the lowest effort. A: impact medium (depends on the buddy), effort low, risk medium (a bad match), reach high, fit medium. B: impact low to medium, effort high, risk medium (cold suggestions can feel forced), reach medium, fit low (engineering-heavy). C: impact medium to high (repeated, structured contact across teams), effort medium, risk low, reach medium, fit high. Recommendation: "I recommend C for the first version, which gives new hires repeated contact with people outside their team by giving up the one-to-one relationship of a buddy; next I'd add buddy matching for people in small offices; I would switch to A if the lunches had low attendance."

Alternative solution. Recommend A plus C, with C as the primary, and say that the combination confounds what you learn.

Tradeoffs. A grid with two high-effort options hides the cheapest test; say that an A-style trial could start next week at almost no cost.

EX-C07-08 · Success metrics for a design

Problem. For a recipe-app feature, "what can I cook with what I have," write a primary metric, one input metric, one guardrail and one counter metric, each defined precisely.

Context. The feature takes a list of ingredients and suggests recipes. The team wants to know if it works.

Data. Invented baseline: 22% of users who open the app on a weeknight afternoon cook a recipe from it that evening.

Constraints. Five minutes.

Expected approach. Use the compact metric template (FW-C05-06) and tie the primary metric to the problem.

Solution. Primary: the share of weeknight sessions with a "what I have" search in which the user marks a recipe cooked within three hours, against the baseline of 22%. Input: the share of searches that return at least one recipe using 80% or more of the listed ingredients. Guardrail: the share of searches returning nothing, below 5% (invented). Counter metric: the rate of "I had to buy something extra" taps, because a feature that suggests recipes requiring a store trip fails its own purpose.

Alternative solution. Use recipes saved instead of recipes cooked as a leading signal, with a note that saving is a weaker proxy.

Tradeoffs. "Marked cooked" depends on users reporting it; a photo prompt or a shopping integration would be stronger and more intrusive.

EX-C07-09 · A timed full run

Problem. Give a 25-minute answer to: "Design a product to help people who are learning to drive practice between lessons."

Context. Practice alone is often not allowed; learners rely on a family member or an instructor's lesson time. This is a self-scored exercise; record yourself.

Data. None. You may assume a country, a platform and a company; say them aloud.

Constraints. Use the short-version budget in section 4: one user, one problem, two solutions, a recommendation, one metric, one risk.

Expected approach. Use the full spine (FW-C07-01), then score yourself against the five behaviors in section 1.

Solution. An outline of a strong answer. Clarify: a phone and in-car accessory feature, goal that learners reach test readiness with more confidence. User: learners with a supervising adult who is not a professional teacher (the supervisor is a second user). Pain points: the supervisor does not know what to practice or how to give feedback; the learner repeats comfortable routes; fear after a bad lesson. Choose the first: practice that is unstructured. Solutions: Assist (a practice plan that suggests a route and a skill for each session, from a phone mount, with the supervisor's prompts in plain language) and Reframe (a post-drive review that replays three moments).

Recommend the practice plan. Metric: the share of learners who complete a planned set of skills before their test. Risk: safety, since a phone near a new driver is a distraction. The design answer is no phone use by the learner, prompts only to the supervisor and audio at stops. Change my mind if learners say the problem is anxiety, not structure.

Alternative solution. Choose the learner's anxiety as the problem and design a calm-down routine, with the same checks.

Tradeoffs. A plan makes practice systematic and may reduce the learner's own judgment; the safety risk is high enough that a reasonable answer might choose a different problem.

EX-C07-10 · Handling interventions

Problem. Write a one- to two-sentence reply to each interviewer intervention: (a) "Who is the user?" (b) "Why that problem?" (c) "What else could you do?" (d) "What could go wrong?" (e) "Let's go straight to solutions."

Context. You are partway through a design answer.

Data. Use any prompt you like; examples below use a pet-adoption product.

Constraints. Two minutes.

Expected approach. Use the table of interventions in section 4: each one points to a missing step.

Solution. (a) "First-time adopters in their first two weeks. I chose them by pain and by fit with our services, and I gave up families who already own pets." (b) "It happens to nearly every adopter, costs them a vet visit or a night of worry, and they already work around it by searching forums. I'd confirm that with interviews." (c) "A different kind of solution: instead of a guide, I could connect them with a mentor who has just adopted. It is slower to scale and may help more." (d) "Misuse and safety: unverified advice about pet health could do

harm, so I'd restrict medical guidance to vet-reviewed content and route emergencies to a vet." (e) "Sure. The problem I'm solving is the first-two-weeks uncertainty, and my first idea is a daily checklist with vet-reviewed answers."

Alternative solution. Record yourself replying, and check that each reply is under 20 seconds and returns to the spine.

Tradeoffs. Short replies keep the answer moving and can skip nuance; offer more if the interviewer asks.

13. Practice protocol

Timed drills (three a week for four weeks).

- *Week 1: user and problems.* Take a prompt from the exercises or the questions (IQ-C07-01, IQ-C07-02, EX-C07-04) and spend ten minutes on the goal, the user, and a ranked pain-point table. Say the criterion aloud.
- *Week 2: range and recommendation.* Use IQ-C07-03, IQ-C07-04 or EX-C07-03. Produce three solutions that differ in kind, fill the grid, and say the recommendation line with a stated cost. Twelve minutes.
- *Week 3: surfaces and risks.* Use IQ-C07-05, IQ-C07-06 or EX-C07-05 and EX-C07-06. Describe moments for a watch, a speaker or a physical device, then run the responsible-design check and state defaults and a launch condition. Fifteen minutes.
- *Week 4: full rounds.* Run two 45-minute answers and one 25-minute answer (EX-C07-09) on prompts you have not seen, with a partner who uses the interventions in section 4.

With a partner. Ask your partner to interrupt with "Who is the user?" and "What could go wrong?" at random points, and to challenge your chosen user once with "Why not [other segment]?" (as in IQ-C07-07). Practice answering in under a minute and returning to the spine.

Record and review. Record every full run. Listen once for structure, once for wording. Count how many times you described a screen when you could have described a moment, and how many solutions were variants of one idea.

Feedback checklist (yes, partly, no).

1. Did I clarify and state a goal before designing?
2. Did I choose a specific user by a stated criterion, and say what I gave up?
3. Did I rank the pain points and choose with a reason?
4. Were my solutions different in kind, each linked to the problem?
5. Did I name the deciding criterion and recommend one option with its cost?
6. Did I describe the first version as moments, and say what comes next?
7. Did the primary metric match the chosen problem, with a guardrail and a counter metric?
8. Did I name two or three harms with a design response and a default?

9. Did I say what would change my mind?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the prompt, the user, the chosen problem, the recommendation, your score out of ten and the step you rushed. After eight drills, look for the step you skip most often (usually the risks or the metrics) and drill that step alone for a week.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Choose a feature your team shipped or is building. Write the user (a segment with a criterion), the pain point it addresses (observable, with the workaround), and the goal ladder (FW-C07-02). Ask a teammate whether they would have written the same problem.

Week 1. Write a one-page design brief for an upcoming piece of work: the user, the ranked pain points, three solutions that differ in kind, the evaluation grid, the recommendation with its stated cost, and the metric with a guardrail and a counter metric. Share it with your designer and an engineer before a solution is chosen, and record what changes.

Week 2. Run the responsible-design check (FW-C07-06) on one feature in development. List the two or three most serious risks, the defaults, and the launch condition. Bring them to the review, and note which became requirements.

Week 2. Watch three people use your product for a task you care about, without helping. Write down each moment of hesitation and each workaround. Compare them with the pain points you had ranked.

Week 3. Take a decision your team made recently and write the choice as an option grid, with the options that were not chosen. Ask the team whether the grid describes the discussion, and what was missing.

Deliverables that become stories: the design brief (a time you framed a problem and chose a solution with a stated cost), the responsible-design check (a time you spotted a risk early and changed a design), and the observation notes (a time you learned from watching users and changed a priority). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. A design answer is a chain: a goal, a specific user chosen by a stated criterion, a ranked pain point, solutions that differ in kind, one recommendation with its cost, and a way to measure it.

2. Write pain points as something you could watch happen, and rank them by frequency, severity and what people do about them today.
3. Describe moments, not screens, and fit the design to the surface: what the device senses, what it can compute locally, how much attention it has, and what cannot change after shipping.
4. Run the responsible-design check on every answer: name two or three harms for this product, a concrete design response and a protective default, and say when you would not launch.
5. Finish with a decision and a way to be wrong: the recommendation, its sacrifice, the metric with a counter metric, and what would change your mind.

Further reading

- Don Norman, *The Design of Everyday Things* (revised and expanded edition, 2013). A foundation for thinking about how people interact with products, including the idea that errors are often design failures.
- Kat Holmes, *Mismatch: How Inclusion Shapes Design* (2018). Inclusive design, and the idea that mismatches between people and products create exclusion. Related Microsoft materials on inclusive design are available from the company's design site [V: MIT Press, 16 October 2018, ISBN 9780262038881].
- Apple, *Human Interface Guidelines* (online, updated regularly). Public design guidance for Apple's platforms, useful for the vocabulary of devices and context [V: Apple Human Interface Guidelines].
- Marty Cagan, *Inspired* (2nd edition, 2017). Product discovery and the role of the product manager in choosing the problem.
- Jake Knapp with John Zeratsky and Braden Kowitz, *Sprint* (2016). A five-day method for defining a problem, sketching solutions and testing with users.
- Gayle Laakmann McDowell and Jackie Bavaro, *Cracking the PM Interview* (2013). A widely used guide with chapters on product design questions [V: CareerCup, first edition, 2 December 2013, ISBN 9780984782819].
- Apple, "Apple Presents iPod," press release (23 October 2001) and "Apple Unveils New iPods" (17 July 2002), on Apple's newsroom archive; Apple's Form 10-K for fiscal 2003 and 2004 (SEC) for unit sales. The primary sources for CS-C07-02.